

Università degli Studi di Salerno  
Laurea Magistrale in Informatica

Corso di Ingegneria del Software: Tecniche Avanzate  
Prof. Andrea De Lucia

# **Software Evolution Report**

## MARK 2.0 Plus

|                  |            |
|------------------|------------|
| Cerchia Giovanni | NF22500202 |
| Medica Vincenzo  | NF22500203 |

<https://github.com/g-cer/mark-ml-classifier>

Anno Accademico 2025/2026  
24/01/2026

# Indice

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Glossario</b>                                                               | <b>ii</b> |
| <b>1 Introduzione</b>                                                          | <b>1</b>  |
| 1.1 Contesto e motivazione . . . . .                                           | 1         |
| 1.2 Obiettivo del lavoro . . . . .                                             | 1         |
| 1.3 Il processo seguito e la struttura del documento . . . . .                 | 1         |
| <b>2 Il sistema di partenza: MARK 2.0</b>                                      | <b>2</b>  |
| 2.1 Regole di classificazione . . . . .                                        | 2         |
| 2.2 Architettura modulare . . . . .                                            | 2         |
| 2.3 Flusso principale di utilizzo . . . . .                                    | 7         |
| 2.4 Input e output . . . . .                                                   | 8         |
| 2.5 Limiti dello stato iniziale . . . . .                                      | 9         |
| <b>3 Verifica della baseline: System Testing pre-modifica</b>                  | <b>9</b>  |
| 3.1 Scomposizione in casi d'uso . . . . .                                      | 9         |
| 3.2 Tipologia e tecnica di testing . . . . .                                   | 10        |
| 3.3 Entry-point per l'esecuzione dei test: <code>main_args.py</code> . . . . . | 10        |
| 3.4 UC-1 — Analisi di una directory di repository . . . . .                    | 10        |
| 3.5 UC-2 — Cloning (e verifica cloning) da lista CSV . . . . .                 | 13        |
| 3.6 Esito della suite pre-modifica . . . . .                                   | 14        |
| <b>4 Le Change Request</b>                                                     | <b>14</b> |
| 4.1 Standard per la classificazione delle Change Request . . . . .             | 15        |
| 4.2 CR1 — Metriche di qualità del codice (Radon) . . . . .                     | 16        |
| 4.3 CR2 — GUI per configurazione ed esecuzione analisi . . . . .               | 17        |
| 4.4 CR3 — Dashboard di reportistica integrata nella GUI . . . . .              | 18        |
| <b>5 Master Test Plan</b>                                                      | <b>19</b> |
| 5.1 Strategia di testing . . . . .                                             | 19        |
| 5.2 Componenti incluse, componenti escluse e coverage . . . . .                | 20        |
| <b>6 Impact Analysis</b>                                                       | <b>21</b> |
| 6.1 Metodo adottato e scelta degli SLO . . . . .                               | 21        |
| 6.2 CR1 — Metriche di qualità del codice . . . . .                             | 21        |
| 6.3 CR2 (GUI) e CR3 (Dashboard) . . . . .                                      | 25        |
| 6.4 Riepilogo . . . . .                                                        | 26        |
| <b>7 Testing post-modifica</b>                                                 | <b>26</b> |
| 7.1 Unit e Integration Testing (white-box) . . . . .                           | 26        |
| 7.2 Post-Modification System Testing . . . . .                                 | 29        |
| 7.3 Regression Testing . . . . .                                               | 35        |
| 7.4 Riepilogo dei risultati . . . . .                                          | 36        |
| <b>8 Conclusioni</b>                                                           | <b>36</b> |
| 8.1 Sintesi dei risultati . . . . .                                            | 36        |
| 8.2 Valutazione del processo . . . . .                                         | 37        |
| 8.3 Limiti e sviluppi futuri . . . . .                                         | 38        |
| <b>9 Riferimenti bibliografici</b>                                             | <b>39</b> |

## Glossario

|                     |                                                                                                                                                 |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CR</b>           | <i>Change Request</i> . Richiesta di modifica al sistema. In questo progetto sono tre: CR1 (metriche di qualità), CR2 (GUI), CR3 (dashboard).   |
| <b>SLO</b>          | <i>Software Life-cycle Object</i> . Unità di granularità su cui si conduce l'Impact Analysis [1]. Qui gli SLO sono i <i>metodi</i> del sistema. |
| <b>SIS</b>          | <i>Starting Impact Set</i> . Insieme degli SLO che si sa già dover essere modificati per realizzare la CR.                                      |
| <b>CIS</b>          | <i>Candidate Impact Set</i> . Insieme degli SLO potenzialmente impattati, stimato prima dell'implementazione a partire dal SIS.                 |
| <b>AIS</b>          | <i>Actual Impact Set</i> . Insieme degli SLO effettivamente modificati o aggiunti dopo l'implementazione.                                       |
| <b>FPIS</b>         | <i>False Positive Impact Set</i> . $CIS \setminus AIS$ : elementi previsti come impattati che non lo sono stati.                                |
| <b>DIS</b>          | <i>Discovered Impact Set</i> . $AIS \setminus CIS$ : elementi impattati che l'analisi non aveva previsto.                                       |
| <b>CC</b>           | <i>Cyclomatic Complexity</i> . Numero di cammini linearmente indipendenti in un blocco di codice [2].                                           |
| <b>MI</b>           | <i>Maintainability Index</i> . Indice sintetico di manutenibilità su scala 0–100 [3].                                                           |
| <b>SLOC</b>         | <i>Source Lines Of Code</i> . Righe di codice non vuote e non costituite da soli commenti; usate come peso nella media dell'MI.                 |
| <b>UC</b>           | <i>Use Case</i> . Caso d'uso da cui si derivano i test di sistema.                                                                              |
| <b>TF</b>           | <i>Test Frame</i> . Combinazione ammissibile di scelte prodotta dalla Category Partition [4].                                                   |
| <b>TC</b>           | <i>Test Case</i> . Istanza eseguibile di un test frame, con input concreti e oracolo.                                                           |
| <b>UT / IT / ST</b> | <i>Unit / Integration / System Test</i> . I tre livelli di verifica previsti dal Master Test Plan.                                              |
| <b>CFG</b>          | <i>Control Flow Graph</i> . Grafo dei cammini di esecuzione di un metodo, usato per la selezione white-box dei casi di test.                    |

# 1 Introduzione

## 1.1 Contesto e motivazione

Negli studi di **Software Engineering for AI-based systems (SE4AI)** è frequente dover costruire dataset affidabili di progetti di Machine Learning a partire da GitHub [5]. La selezione manuale dei repository è però onerosa e **soggetta a bias**, soprattutto quando la classificazione si fonda su descrizioni testuali o su *buzzword* presenti nei README: due progetti che dichiarano la stessa cosa possono fare cose molto diverse, e viceversa.

In tale contesto MARK 2.0 nasce come supporto per i ricercatori, offrendo un tool di classificazione automatica di progetti ML in base alla loro natura d'uso: *addestramento* rispetto a *utilizzo* di modelli. L'approccio adottato è **euristico e interpretabile**: ogni decisione di classificazione è motivata da regole esplicite applicate a elementi osservabili del codice sorgente — le librerie importate e le API ML invocate. La scelta è coerente sia con l'esigenza di **scalabilità** su grandi dataset, sia con la necessità di trasparenza del processo decisionale, che un classificatore appreso non offrirebbe con la stessa immediatezza.

## 1.2 Obiettivo del lavoro

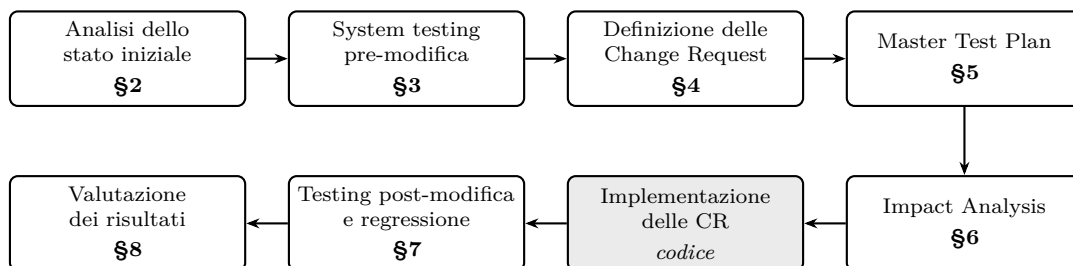
Il presente documento descrive l'evoluzione di MARK 2.0 in MARK 2.0 Plus, realizzata attraverso tre Change Request:

- **CR1** — integrazione di metriche quantitative di qualità del codice (Cyclomatic Complexity e Maintainability Index);
- **CR2** — interfaccia grafica per la configurazione e l'esecuzione delle analisi;
- **CR3** — dashboard di reportistica aggregata sui risultati di una run.

L'obiettivo del lavoro non si esaurisce però nell'implementazione delle tre estensioni. Ciò che il progetto intende mostrare è l'applicazione di un processo di **manutenzione controllata**, in cui ogni modifica è classificata secondo lo standard **ISO/IEC/IEEE 14764:2022** [6], è preceduta da un'analisi di impatto quantificata e verificata a posteriori, ed è validata da un piano di test definito prima dell'implementazione. La baseline viene sottoposta a test di sistema *prima* di essere toccata, così da disporre di un riferimento oggettivo con cui dimostrare, a valle delle modifiche, l'assenza di regressioni.

## 1.3 Il processo seguito e la struttura del documento

Le attività si sono svolte nell'ordine illustrato in figura 1: lo stato iniziale viene prima compreso e documentato, poi verificato; solo a quel punto vengono definite le modifiche, pianificata la loro verifica e stimato il loro impatto; l'implementazione è seguita dal testing post-modifica e dalla riesecuzione della suite di regressione.



**Figura 1:** Il processo di evoluzione seguito e le sezioni che lo documentano

Il documento segue lo stesso ordine. La sezione 2 descrive MARK 2.0 allo stato iniziale — architettura, flusso di esecuzione e limiti che motivano le modifiche. La sezione 3 riporta il system testing condotto sulla baseline prima di qualunque modifica, che produce la suite

poi riutilizzata per la regressione. La sezione 4 definisce e classifica le tre Change Request; la sezione 5 fissa la strategia di verifica; la sezione 6 stima e poi misura l'impatto delle modifiche sui componenti esistenti. La sezione 7 raccoglie il testing successivo all'implementazione — unit, integrazione, sistema e regressione — e la sezione 8 valuta il risultato complessivo del processo.

## 2 Il sistema di partenza: MARK 2.0

MARK 2.0 è un tool di classificazione automatica di repository Python che mira a distinguere i progetti *applied ML* in due categorie:

- **ML-Model Producer**: progetti che costruiscono o addestrano modelli ML;
- **ML-Model Consumer**: progetti che utilizzano modelli ML pre-addestrati (inferenza).

Coerentemente con la finalità di supporto alla ricerca, l'obiettivo è privilegiare una classificazione **affidabile e ad alta precisione**, riducendo al minimo il rischio di includere progetti non pertinenti nel campione selezionato per le analisi successive.

Il tool opera tramite **analisi statica** dei sorgenti:

1. rileva le librerie ML importate (ad es. TensorFlow, scikit-learn, Keras);
2. rileva le invocazioni di API ML nei file, distinguendo quelle di training da quelle di inferenza;
3. applica regole euristiche per etichettare i progetti.

Integra inoltre una fase di supporto alla ricerca che automatizza *(i)* la clonazione da una lista predefinita di progetti GitHub e *(ii)* la classificazione dei progetti clonati.

### 2.1 Regole di classificazione

#### 2.1.1 Classificazione Producer

Un progetto è classificato come **Producer** se contiene almeno un file che:

1. importa almeno una libreria ML del knowledge base;
2. contiene almeno una chiamata a un'API di training (ad es. `.fit()`, `.train()`).

#### 2.1.2 Classificazione Consumer

Un progetto è classificato come **Consumer** se contiene almeno un file che:

1. importa almeno una libreria ML del knowledge base;
2. contiene almeno una chiamata a un'API di inferenza;
3. **non** contiene API di training (*Rule 3*);
4. non presenta nel filename pattern tipici di file di test o valutazione — `test`, `eval`, `example`, `validat` (*Rule 4*).

Le condizioni aggiuntive Rule 3 e Rule 4 servono a ridurre i falsi positivi: tipicamente quando API di inferenza vengono usate solo per valutare modelli addestrati nel progetto stesso, uno scenario che potrebbe far apparire un Producer come Consumer.

**Nota di configurabilità.** La Rule 3 è parametrica: il Facade la espone come argomento `rules_3` di `run_analysis`, così da poter misurare l'effetto della regola sui risultati. Il valore predefinito è *attiva*.

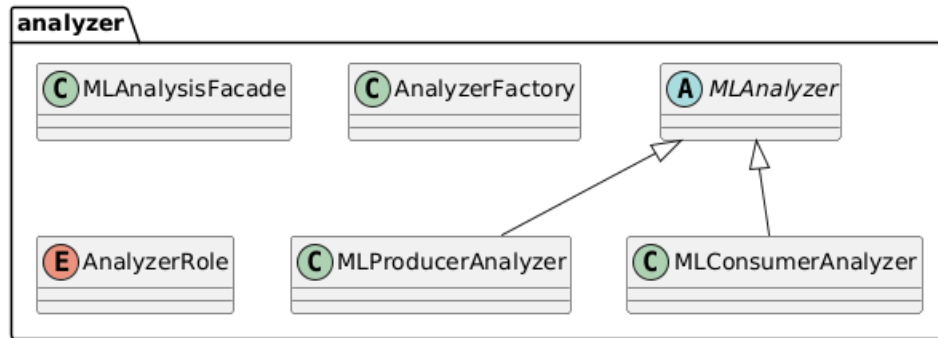
### 2.2 Architettura modulare

Con la versione 2.0 MARK 2.0 è stato ristrutturato come sistema object-oriented, con separazione delle responsabilità in moduli *core* raccolti nella cartella `modules/`:

- `analyzer/` — logica di classificazione;
- `cloner/` — acquisizione dei repository da GitHub;
- `keyword_extractor/` — estrazione delle keyword e delle API ML;

- `library_manager/` — gestione delle librerie Producer/Consumer da identificare;
- `scanner/` — filtraggio dei file dei progetti;
- `utils/` — logger centralizzato;
- `oracle/` — validazione dei risultati e calcolo delle metriche di accuratezza.

### 2.2.1 `modules/analyzer` — Orchestrazione dell'analisi



**Figura 2:** Package `modules/analyzer`

- `MLAnalyzer` (*astratta*): classe base per tutti gli analyzer. Definisce il workflow comune per scansionare i progetti, applicare i filtri configurati, estrarre librerie e keyword ed esportare i risultati in CSV. Realizza il **Template Method**: i metodi di workflow sono fissi, le classi concrete specializzano solo `check_library`.
- `MLProducerAnalyzer`: specializzazione per rilevare progetti che costruiscono o addestrano modelli ML. Applica i filtri di estensione e le strategie di keyword matching per identificare l'uso di API di training e di salvataggio modelli.
- `MLConsumerAnalyzer`: specializzazione per identificare progetti che utilizzano modelli pre-addestrati. Applica le regole consumer sui file sorgente, con supporto opzionale della Rule 3 tramite il parametro `rules_3`.
- `MLAnalysisFacade`: implementa il **Facade** che nasconde la complessità del workflow dietro l'interfaccia semplificata `run_analysis(**kwargs)`. Coordina l'intera pipeline di analisi delegando alla factory la creazione degli analyzer e gestendo l'esecuzione end-to-end.
- `AnalyzerFactory`: implementa un pattern **Factory-Registry** che permette la registrazione dinamica dei builder tramite il decoratore `@register(role)` e l'istanziamento tramite `create_builder(role)`. Mantiene un registro interno delle classi builder associate ai vari ruoli di analisi.
- `AnalyzerRole` (*Enum*): enumerazione che definisce i ruoli disponibili per gli analyzer ML. Nello stato iniziale i ruoli sono PRODUCER e CONSUMER.

### 2.2.2 modules/analyzer/builder — Costruzione configurata degli analyzer

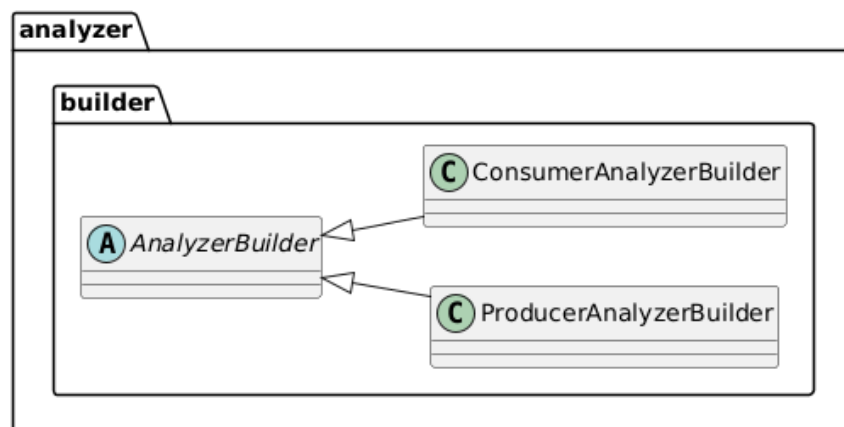


Figura 3: Package modules/analyzer/builder

- `AnalyzerBuilder` (*astratta*): implementa il **Builder** per assemblare istanze di `MAnalyzer` con configurazioni consistenti. Raccoglie passo dopo passo tutte le dipendenze richieste — ruolo, filtri, strategie, dizionari — e produce un analyzer completo solo quando la configurazione è valida. Centralizza la logica di setup, eliminando codice di inizializzazione duplicato e oggetti parzialmente configurati.
- `ConsumerAnalyzerBuilder`: builder concreto pre-configurato per istanziare `MLConsumerAnalyzer`. Registrato automaticamente tramite `@register(AnalyzerRole.CONSUMER)`, configura di default il filtro `.py` e `ExcludeTestFilesFilter`, i dizionari `consumer/producer` e la strategia `DefaultKeywordMatcher`.
- `ProducerAnalyzerBuilder`: builder concreto pre-configurato per istanziare `MLProducerAnalyzer`. Registrato tramite `@register(AnalyzerRole.PRODUCER)`, applica il filtro `.py`, il dizionario `producer` e la strategia `DefaultKeywordMatcher` per rilevare le API di training.

### 2.2.3 modules/cloner — Acquisizione dei progetti

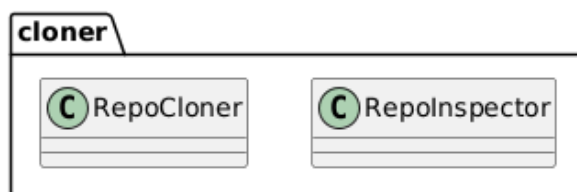


Figura 4: Package modules/cloner

- `RepoCloner`: gestisce la clonazione di massa dei repository GitHub tramite *shallow clone* paralleli con `ThreadPoolExecutor`. Registra successi ed errori su file CSV (`cloned_log.csv`, `errors.csv`) per tracciare gli esiti del processo di acquisizione, e salta i repository già clonati rendendo le esecuzioni idempotenti.
- `RepoInspector`: analizza i risultati del processo di clonazione verificando quali repository sono stati effettivamente clonati. Confronta il file CSV di log con il filesystem e genera report di riconciliazione (`not_cloned_repos.csv`, `effective_repos.csv`).

#### 2.2.4 modules/keyword\_extractor — Estrazione keyword e API

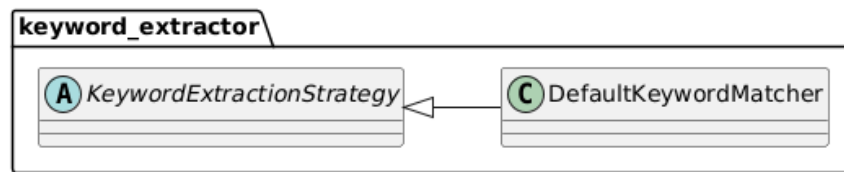


Figura 5: Package modules/keyword\_extractor

- `KeywordExtractionStrategy` (*astratta*): definisce il contratto per l'estrazione di keyword dai file sorgente. Implementa lo **Strategy**, permettendo di sostituire dinamicamente l'algoritmo di matching senza modificare le classi analyzer.
- `DefaultKeywordMatcher`: implementazione concreta dell'estrazione keyword mediante scansione riga per riga con pattern regex. Costruisce espressioni regolari case-insensitive gestendo i caratteri speciali (punti, parentesi) e il whitespace opzionale, restituendo le keyword trovate con il numero di riga corrispondente.

#### 2.2.5 modules/library\_manager — Dizionari e import ML

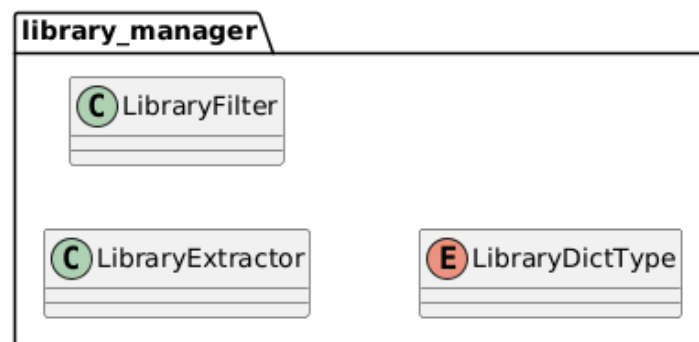


Figura 6: Package modules/library\_manager

- `LibraryExtractor`: utility per identificare gli statement di import nei file sorgente Python. Esegue matching testuale riga per riga su pattern `import` e `from...import`, estraendo i nomi delle librerie utilizzate nel codice.
- `LibraryFilter`: fornisce metodi per caricare i dizionari di librerie ML da file CSV e filtrare quelle effettivamente utilizzate nei progetti. Confronta le librerie importate con quelle presenti nei dizionari consumer e producer.
- `LibraryDictType` (*Enum*): enumerazione che identifica i tipi di dizionario disponibili (`CONSUMER`, `PRODUCER`) per categorizzare le librerie ML in base al loro utilizzo tipico.



### 2.2.6 modules/scanner — Filtraggio e scansione file

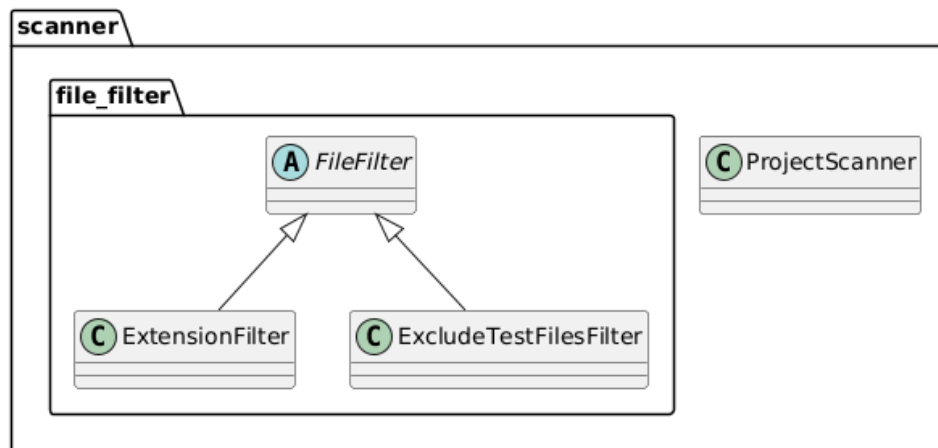


Figura 7: Package modules/scanner

- **FileFilter** (*astratta*): classe base per implementare filtri di selezione dei file durante la scansione. Definisce il metodo `accepts(file_path)` che le classi concrete specializzano.
- **ExtensionFilter**: accetta esclusivamente file con estensioni specifiche (ad es. `.py`, `.ipynb`), limitando l'analisi a determinati tipi di file sorgente.
- **ExcludeTestFilesFilter**: scarta i file comunemente associati a testing, esempi, valutazioni o validazioni sulla base di pattern nel nome file. Implementa la Rule 4 dei Consumer, escludendo codice non rappresentativo dell'utilizzo reale.
- **ProjectScanner**: coordina la scansione ricorsiva delle cartelle di progetto per raccogliere i file sorgente che soddisfano tutti i filtri configurati, applicandoli in sequenza.

### 2.2.7 modules/oracle e modules/utils — Validazione e logging

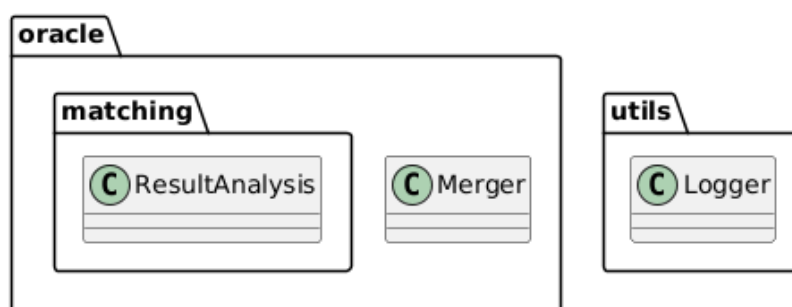


Figura 8: Package modules/oracle e modules/utils

- **Merger**: compara i risultati dell'analisi con un file oracolo di *ground truth* e calcola le metriche di valutazione (Precision, Recall, F1-Score, Accuracy). Esporta falsi positivi e falsi negativi in file CSV separati per l'analisi degli errori.
- **ResultAnalysis**: confronta i risultati ottenuti dai singoli progetti analizzati con il file oracolo di riferimento. Aggrega le metriche a livello di progetto ed esporta un report CSV con tutti i valori di performance per ogni repository.

- `get_logger()`: funzione factory che crea e restituisce un'istanza configurata di logger Python. Il logger è utilizzato trasversalmente da tutti i moduli per garantire tracciabilità uniforme di operazioni, errori e performance del sistema.

## 2.3 Flusso principale di utilizzo

Nello stato iniziale MARK 2.0 è eseguibile tramite una pipeline avviata dallo script `main.py`, che ne raccoglie i parametri di configurazione.

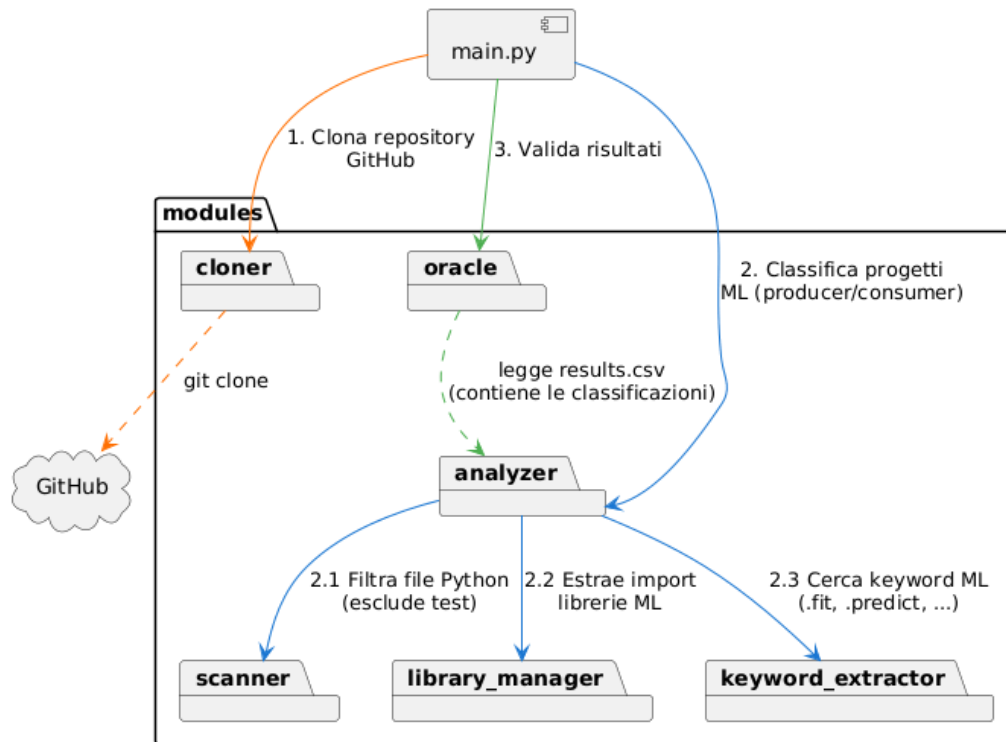


Figura 9: Flusso dei componenti a partire da `main.py`

### Step 1 — Clonazione dei repository

RepoCloner:

1. legge un CSV con la lista dei progetti GitHub;
2. clona in parallelo con `ThreadPoolExecutor`;
3. genera la struttura `io/repos/<user>/<project>/...`;
4. `RepoInspector` verifica la consistenza del file CSV con il filesystem.

### Step 2 — Analisi ML (classificazione)

`MLAnalysisFacade` coordina un `AnalyzerFactory` per costruire un `MLAnalyzer` specializzato; per ogni progetto:

1. filtra i file (ad es. `.py`) con `ProjectScanner`;
2. estrae e filtra le librerie con `LibraryExtractor` e `LibraryFilter`;
3. trova keyword e API con `DefaultKeywordMatcher`;
4. produce `results.csv` con una riga per evidenza rilevata (progetto, libreria, file, keyword/API, numero di riga).

Tabella 1: Struttura di una riga di `results.csv`

| ProjectName | Is ML producer | libraries  | where     | keyword | line_number |
|-------------|----------------|------------|-----------|---------|-------------|
| user/repo-A | Yes            | tensorflow | /model.py | .fit    | 45          |

### Step 3 — Validazione con oracolo (se attivato)

Merger:

1. calcola TP/FP/TN/FN e le metriche di classificazione;
2. genera `false_positives.csv` e `false_negatives.csv`.

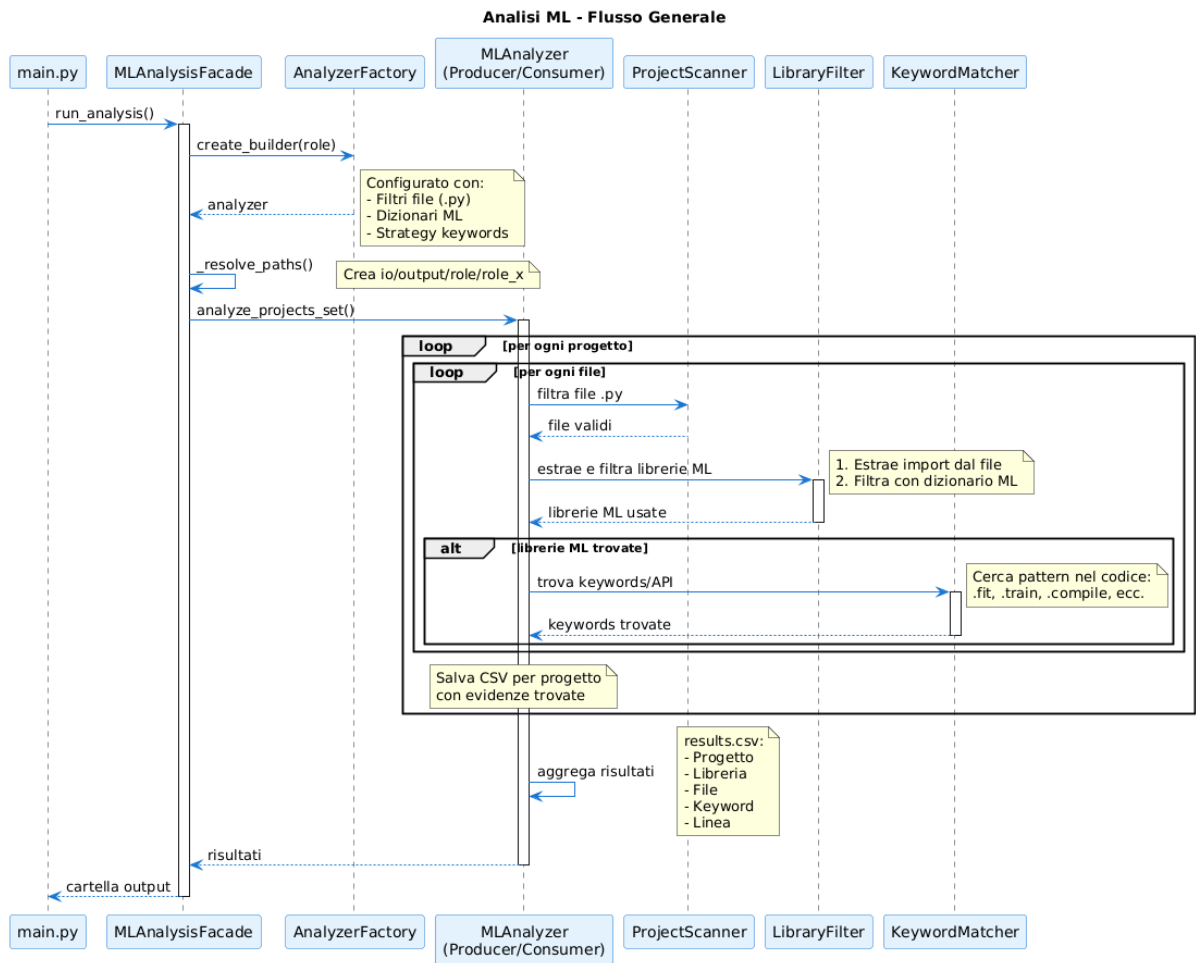


Figura 10: Sequence diagram dello Step 2 — Analisi ML

## 2.4 Input e output

### Input principali

- CSV contenente la lista di repository GitHub, se si decide di utilizzare il cloner;
- cartella contenente i progetti da analizzare, se si decide di eseguire i soli step di analisi;
- *knowledge base*: dizionari in formato `.csv` di librerie ML e API (training/inferenza), utilizzata per filtrare import e chiamate API.

## Output principali

- **Output cloner:** `cloned_log.csv`, `errors.csv`, `not_cloned_repos.csv`, `effective_repos.csv`.
- **Output analisi:** `user_repo_ml_producer.csv` o `user_repo_ml_consumer.csv` per ogni singolo progetto classificato, e `results.csv` con i risultati uniti secondo lo schema di tabella 1.
- **Output validazione:** metriche di Precision, Recall, F1 e Accuracy, con `false_positives.csv` e `false_negatives.csv`.

## 2.5 Limiti dello stato iniziale

Lo stato appena descritto costituisce la **baseline** del lavoro. L'analisi del sistema ne ha evidenziati tre limiti, riassunti in tabella 2: sono questi a motivare, uno per uno, le tre Change Request definite nella sezione 4.

**Tabella 2:** Limiti dello stato iniziale e Change Request corrispondente

| ID | Limite                                      | Situazione attuale (As-Is)                                                                                                                                                                                                        | CR  |
|----|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| L1 | Assenza di indicatori di qualità del codice | Il sistema produce output relativi a librerie, keyword e classificazione (ad es. <code>results.csv</code> ), ma non calcola né esporta alcuna metrica di complessità o manutenibilità a livello di progetto.                      | CR1 |
| L2 | Configurazione <i>hard-coded</i>            | I parametri della pipeline sono impostati direttamente nello script <code>main.py</code> : eseguire più run con configurazioni diverse richiede interventi manuali sul codice, e l'uso del tool resta orientato a utenti tecnici. | CR2 |
| L3 | Risultati consultabili solo come file       | I risultati sono prodotti come CSV per run e per progetto, senza alcuna vista aggregata: l'interpretazione su più repository è demandata alla lettura manuale dei file.                                                           | CR3 |

Prima di intervenire su questi limiti, però, la baseline è stata sottoposta a verifica: è l'oggetto della sezione seguente.

## 3 Verifica della baseline: System Testing pre-modifica

Prima dell'applicazione di qualunque Change Request il sistema è stato sottoposto a una suite di **System Testing**. L'obiettivo è duplice: verificare che la baseline si comporti come specificato e, soprattutto, costruire una **base riutilizzabile** per il Regression Testing post-modifica (sezione 7.3).

### 3.1 Scomposizione in casi d'uso

Per eseguire il system testing in modo più efficace e controllabile, l'utilizzo del sistema è stato scomposto in due funzionalità / casi d'uso separati:

- **UC-1 (Analysis):** analisi e classificazione di una directory locale contenente repository;
- **UC-2 (Cloning):** clonazione di repository Git da una lista CSV.

**Nota.** L'analisi (UC-1) può essere eseguita **anche senza cloning**, a condizione di disporre di una directory con repository già presenti localmente. È questa indipendenza a rendere i due casi d'uso testabili separatamente.

## 3.2 Tipologia e tecnica di testing

La tecnica scelta è il **testing funzionale** / **black-box**, che consente di verificare il comportamento esterno del sistema rispetto ai requisiti specificati, senza analizzarne l'implementazione interna.

Per la progettazione dei casi di test è stato adottato il metodo **Category Partition** [4], una strategia che permette di:

- individuare i **parametri** di input e gli **oggetti dell'ambiente** rilevanti per ogni caso d'uso;
- suddividere ciascun parametro in **categorie** e **scelte** (classi di equivalenza);
- generare **test frame** validi, ciascuno rappresentante un insieme coerente di scelte;
- derivare da questi i **test case** eseguibili e i rispettivi **oracoli** (risultati attesi).

È la stessa tecnica adottata più avanti per il system testing delle nuove funzionalità (sezione 7.2), dove non viene ridescritta.

## 3.3 Entry-point per l'esecuzione dei test: `main_args.py`

Allo stato iniziale l'avvio era disponibile solo tramite `main.py`, ma diversi parametri risultavano configurati tramite variabili *hard-coded* (limite L2 di tabella 2), rendendo poco pratico eseguire più scenari in modo ripetibile senza modifiche manuali al codice.

Per questo motivo, per l'attività di system testing è stata introdotta una variante dell'entry-point, denominata `main_args.py`, invocabile da terminale con argomenti CLI, così da selezionare i parametri necessari ai test mantenendo invariato il codice sorgente.

```
1 python main_args.py --repository-path <dir> --analysis
2 python main_args.py --project-list <csv> --clone --clone-check
```

**Listing 1:** Invocazione tipica usata dai test di sistema

## 3.4 UC-1 — Analisi di una directory di repository

**Tabella 3:** UC-1 — Analysis

|                                    |                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrizione</b>                 | L'utente esegue solo l'analisi su una directory locale contenente repository già disponibili, ottenendo i risultati in <code>io/output/</code> .                                                      |
| <b>Attori</b>                      | Utente (ricercatore)                                                                                                                                                                                  |
| <b>Entry condition</b>             | <code>main_args.py</code> disponibile; dipendenze installate; directory <code>--repository-path</code> accessibile (per scenari "success").                                                           |
| <b>Exit condition</b>              | <i>Successo:</i> risultati salvati in <code>io/output/</code> .<br><i>Errore:</i> messaggio d'errore e assenza di output significativo.                                                               |
| <b>Flusso di eventi principale</b> | 1. Invocazione con <code>--repository-path</code> e <code>--analysis</code> .<br>2. Validazione path.<br>3. Scansione e analisi dei repository.<br>4. Salvataggio output in <code>io/output/</code> . |

**Parametri:** path della directory contenente i repository (`repository_path`).

**Oggetti dell'ambiente:** filesystem, contenuto dei progetti appartenenti alla directory analizzata.

### 3.4.1 Categorie e scelte

**Tabella 4:** UC-1 — Categorie e scelte

| Categoria                 | ID  | Descrizione                      | Proprietà / Vincoli                          |
|---------------------------|-----|----------------------------------|----------------------------------------------|
| Esistenza input directory | ID1 | Directory non esiste             | [Error]                                      |
|                           | ID2 | Directory esiste                 | [property DirOk]                             |
| Contenuto directory       | CI0 | Directory vuota (0 progetti)     | [if DirOk] [property Empty]                  |
|                           | CI1 | Directory con 1 progetto         | [if DirOk] [property NonEmpty]               |
|                           | CI2 | Directory con più progetti (> 1) | [if DirOk] [property NonEmpty, MultiProject] |
| Cardinalità Producer      | NP0 | 0 producer                       | [if DirOk]                                   |
|                           | NP1 | 1 producer                       | [if NonEmpty]                                |
|                           | NP2 | 2+ producer                      | [if MultiProject] [Single]                   |
| Cardinalità Consumer      | NC0 | 0 consumer                       | [if DirOk]                                   |
|                           | NC1 | 1 consumer                       | [if NonEmpty]                                |
|                           | NC2 | 2+ consumer                      | [if MultiProject] [Single]                   |

### 3.4.2 Test Frame

**Tabella 5:** UC-1 — Test frame

| ID   | Combinazioni (categorie/scelte) | Oracolo (risultato atteso)                                       |
|------|---------------------------------|------------------------------------------------------------------|
| TF1  | ID1                             | Messaggio di errore “Input folder not found”                     |
| TF2  | ID2, CI0, NP0, NC0              | Nessun progetto classificato                                     |
| TF3  | ID2, CI1, NP0, NC0              | Progetto non classificato                                        |
| TF4  | ID2, CI1, NP1, NC0              | Progetto classificato “Producer”                                 |
| TF5  | ID2, CI1, NP0, NC1              | Progetto classificato “Consumer”                                 |
| TF6  | ID2, CI1, NP1, NC1              | Progetto classificato “Producer + Consumer”                      |
| TF7  | ID2, CI2, NP0, NC0              | Nessun progetto classificato                                     |
| TF8  | ID2, CI2, NP1, NC0              | Almeno un progetto classificato “Producer”                       |
| TF9  | ID2, CI2, NP0, NC1              | Almeno un progetto classificato “Consumer”                       |
| TF10 | ID2, CI2, NP1, NC1              | Almeno 1 “Producer” e 1 “Consumer”, oppure 1 “Producer+Consumer” |
| TF11 | ID2, CI2, NP2, NC1              | 2+ progetti classificati “Producer”                              |
| TF12 | ID2, CI2, NP1, NC2              | 2+ progetti classificati “Consumer”                              |

### 3.4.3 Test Case

**Tabella 6:** UC-1 — Test case

| TC   | Input<br>(repository_path) | Descrizione<br>ambiente                      | Oracolo                                     |
|------|----------------------------|----------------------------------------------|---------------------------------------------|
| TC1  | repo_does_not_exist        | Directory inesistente                        | Messaggio “Input folder not found”          |
| TC2  | empty_repo                 | Cartella vuota                               | Nessun progetto classificato                |
| TC3  | test_repos                 | 1 progetto senza librerie ML                 | Nessun progetto classificato                |
| TC4  | test_repos                 | 1 progetto con funzioni di training          | Progetto classificato “Producer”            |
| TC5  | test_repos                 | 1 progetto con funzioni di inferenza         | Progetto classificato “Consumer”            |
| TC6  | test_repos                 | 1 progetto con training e inferenza separati | Progetto classificato “Producer + Consumer” |
| TC7  | test_repos                 | Più progetti, nessuno ML                     | Nessun progetto classificato                |
| TC8  | test_repos                 | Più progetti, almeno uno con training        | Almeno un progetto “Producer”               |
| TC9  | test_repos                 | Più progetti, almeno uno con inferenza       | Almeno un progetto “Consumer”               |
| TC10 | test_repos                 | Più progetti con training e inferenza        | Almeno un “Producer” e un “Consumer”        |
| TC11 | test_repos                 | $\geq 2$ progetti con training               | Almeno due “Producer”                       |
| TC12 | test_repos                 | $\geq 2$ progetti con inferenza              | Almeno due “Consumer”                       |

**Implementazione.** I dodici test case sono implementati in `test/system_testing/analysis_test/analysis_test.py`; gli ambienti descritti sopra corrispondono alle directory versionate sotto `analysis_test/test_repos/` (TF3 ... TF12).

### 3.5 UC-2 — Cloning (e verifica cloning) da lista CSV

**Tabella 7:** UC-2 — Cloning

|                                    |                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrizione</b>                 | L'utente clona i repository elencati in un CSV e verifica gli esiti, salvando i repository in <code>io/repos/</code> e producendo log e report.                                                                                                                                                    |
| <b>Attori</b>                      | Utente (ricercatore)                                                                                                                                                                                                                                                                               |
| <b>Entry condition</b>             | <code>main_args.py</code> disponibile; dipendenze installate; CSV disponibile (per gli scenari di successo); accesso a rete/Git; permessi di scrittura in <code>io/repos/</code> .                                                                                                                 |
| <b>Exit condition</b>              | <i>Successo</i> : repo clonati in <code>io/repos/</code> e report disponibili.<br><i>Errore</i> : messaggio d'errore e cloning non eseguito o parziale.                                                                                                                                            |
| <b>Flusso di eventi principale</b> | <ol style="list-style-type: none"><li>1. Invocazione con <code>--project-list</code>, <code>--clone</code>, <code>--clone-check</code>.</li><li>2. Validazione CSV.</li><li>3. Clonazione repo in <code>io/repos/</code>.</li><li>4. Verifica cloning e report.</li><li>5. Terminazione.</li></ol> |

**Parametri:** path del file CSV contenente l'elenco di repository (`project_list_path`).

**Oggetti dell'ambiente:** filesystem, contenuto del file CSV specificato.

#### 3.5.1 Categorie e scelte

**Tabella 8:** UC-2 — Categorie e scelte

| Categoria          | ID  | Descrizione                | Vincoli                |
|--------------------|-----|----------------------------|------------------------|
| Esistenza file CSV | EF1 | File CSV non esiste        | [Error]                |
|                    | EF2 | File CSV esiste            | [property file_exists] |
| Contenuto CSV      | CC1 | CSV vuoto (solo header)    | [if file_exists]       |
|                    | CC2 | CSV con 1 repository       | [if file_exists]       |
|                    | CC3 | CSV con N repository (> 1) | [if file_exists]       |

#### 3.5.2 Test Frame

**Tabella 9:** UC-2 — Test frame

| ID  | Combinazioni | Oracolo                                              |
|-----|--------------|------------------------------------------------------|
| TF1 | EF1          | Nessun repository clonato (errore: file inesistente) |
| TF2 | EF2, CC1     | Nessun repository clonato (CSV vuoto)                |
| TF3 | EF2, CC2     | 1 repository clonato correttamente                   |
| TF4 | EF2, CC3     | N repository clonati correttamente                   |



### 3.5.3 Test Case

**Tabella 10:** UC-2 — Test case

| TC  | Input (project_list_path) | Descrizione ambiente                                     | Oracolo                                                  |
|-----|---------------------------|----------------------------------------------------------|----------------------------------------------------------|
| TC1 | non-existent.csv          | File CSV inesistente                                     | Messaggio “Project list file not found”                  |
| TC2 | empty_projects.csv        | File CSV vuoto                                           | Nessun progetto clonato                                  |
| TC3 | single_project.csv        | Contiene 10up/classifai                                  | Il progetto <i>classifai</i> viene clonato correttamente |
| TC4 | multi_projects.csv        | Contiene 10up/classifai, 2kna1/Honesty, 5hirish/adam_qas | Tutti i repository vengono clonati correttamente         |

**Implementazione.** I quattro test case sono implementati in `test/system_testing/cloning_test/cloning_test.py`; i file CSV di input sono versionati sotto `cloning_test/test_data/`.

### 3.6 Esito della suite pre-modifica

Il testing di sistema effettuato secondo la tecnica **Category Partition** ha permesso di esplorare in modo sistematico tutte le combinazioni significative dei parametri di input e dello stato degli oggetti dell’ambiente, garantendo la copertura funzionale delle due principali funzionalità del tool: (i) analisi e classificazione dei progetti ML ed (ii) clonazione dei repository da file CSV.

Complessivamente sono stati definiti, implementati ed eseguiti **16 test case** — 12 per UC-1 *Analysis* e 4 per UC-2 *Cloning* — e l’esecuzione è terminata **senza alcun fallimento**: tutti i test case hanno soddisfatto gli oracoli attesi.

**Tabella 11:** Esito della suite pre-modifica

| Caso d’uso    | Test case | Esito               |
|---------------|-----------|---------------------|
| UC-1 Analysis | 12        | <b>12 pass</b>      |
| UC-2 Cloning  | 4         | <b>4 pass</b>       |
| <b>Totale</b> | <b>16</b> | <b>0 fallimenti</b> |

Questa base di test viene riutilizzata **integralmente** come suite di regressione nella sezione 7.3, per verificare che le modifiche introdotte dalle Change Request non compromettano il comportamento corretto del sistema.

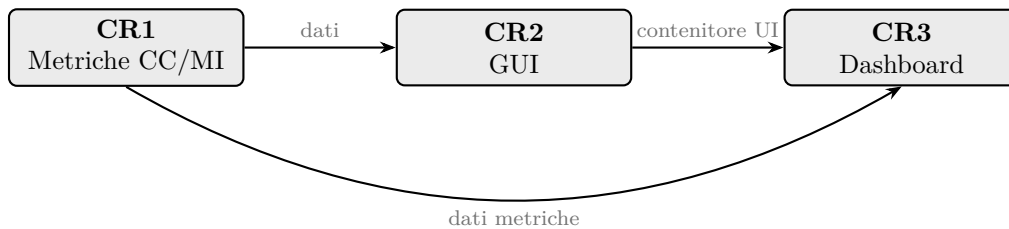
## 4 Le Change Request

Verificata la baseline, l’evoluzione del sistema è stata definita attraverso tre Change Request, ciascuna rivolta a uno dei limiti individuati in tabella 2.

**Tabella 12:** Le tre Change Request

| ID  | Titolo                               | Sintesi                                                                                                                             | Tipo                                | Stato             |
|-----|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-------------------|
| CR1 | Metriche di qualità del codice       | Introduzione del calcolo di metriche di qualità tramite la libreria Radon: Cyclomatic Complexity (CC) e Maintainability Index (MI). | Enhancement (Additive + Perfective) | <b>Rilasciata</b> |
| CR2 | GUI per configurazione ed esecuzione | Introduzione di una GUI (Tkinter) per configurare ed eseguire analisi e cloning senza modifiche manuali a script e variabili.       | Enhancement (Additive)              | <b>Rilasciata</b> |
| CR3 | Dashboard di reportistica            | Introduzione di una dashboard (matplotlib embedded nella GUI) con grafici e statistiche aggregate per run.                          | Enhancement (Additive + Perfective) | <b>Rilasciata</b> |

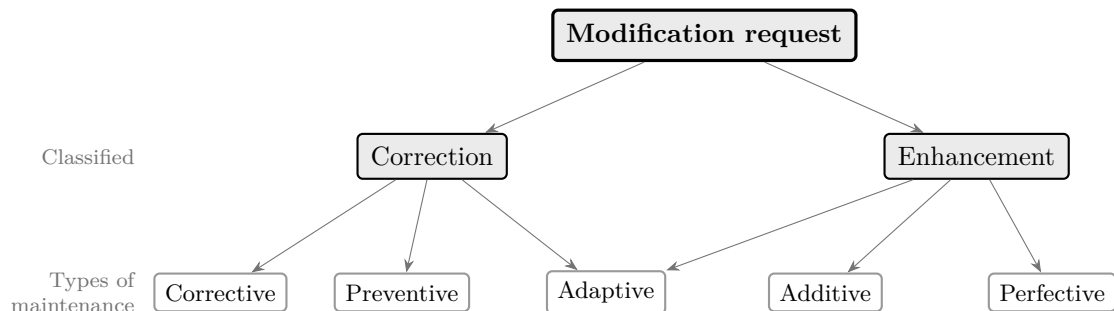
Le tre modifiche non sono indipendenti: **CR2** sfrutta e visualizza le informazioni introdotte da **CR1**, quando disponibili, mentre **CR3** dipende logicamente sia da CR1 (dati delle metriche) sia da CR2 (contenitore UI e gestione delle run).



**Figura 11:** Dipendenze logiche tra le Change Request

#### 4.1 Standard per la classificazione delle Change Request

Secondo lo standard **ISO/IEC/IEEE 14764:2022** [6], una **Modification Request (MR)** è un item informativo che identifica e descrive cambiamenti proposti al prodotto o servizio. Una MR può essere classificata secondo lo schema di figura 12.



**Figura 12:** Classificazione di una Modification Request secondo ISO/IEC/IEEE 14764:2022 [6]

In particolare, una **Correction** è un cambiamento che risolve un problema e colma un gap per rendere il software operativo rispetto ai requisiti operativi definiti. Un **Enhancement** è un cambiamento che implementa un **nuovo requisito**; lo standard precisa che un enhancement *non* è una correction.

Dopo la classificazione principale, la MR può essere identificata anche con una *tipologia di manutenzione*:

- **Adaptive maintenance** (collegabile a correction ed enhancement): modifica per mantenere il prodotto usabile in un ambiente cambiato o in costante cambiamento;
- **Additive maintenance** (enhancement): modifica post-delivery per **aggiungere funzionalità o feature**;
- **Perfective maintenance** (enhancement): modifica per fornire miglioramenti agli utenti in termini di comprensibilità e informazione, e/o per migliorare performance, manutenibilità o altri attributi.

Ogni CR riporta di seguito la **classificazione** (Enhancement o Correction) e la **tipologia di manutenzione** con una breve motivazione.

## 4.2 CR1 — Metriche di qualità del codice (Radon)

### Motivazione

Nella versione attuale, MARK 2.0 produce risultati di classificazione ML (Producer/Consumer) ma non include indicatori quantitativi di qualità del codice del progetto (limite L1 di tabella 2). In contesti di analisi su dataset di repository, poter associare ai risultati anche misure come complessità e manutenibilità facilita:

- l'interpretazione dei risultati;
- il confronto tra repository;
- la selezione di progetti “più manutenibili” per analisi successive.

### Situazione desiderata (To-Be)

Integrare nella pipeline il calcolo di metriche tramite **Radon**, in particolare:

- **Cyclomatic Complexity (CC)** [2];
- **Maintainability Index (MI)** [3].

Le metriche devono essere calcolate sui file sorgente analizzati e aggregate a livello di progetto.

### Descrizione della modifica

- Durante l'analisi dei file, oltre all'estrazione di librerie e keyword, il sistema calcola CC e MI.
- I valori delle metriche vengono raccolti e aggregati a livello di progetto secondo le formule di equazioni (1) e (2): la CC è calcolata per ogni blocco di codice presente nei file Python e mediata aritmeticamente su tutti i blocchi; l'MI è calcolato per ciascun file Python e poi mediato in modo **pesato sullo SLOC** di ogni file, ottenendo una media complessiva rappresentativa dell'intero progetto.
- Le metriche vengono salvate in output in modo consultabile e allineato alla run corrente.

$$\overline{CC}_p = \frac{1}{|B_p|} \sum_{b \in B_p} CC(b) \quad (1)$$

$$\overline{MI}_p = \frac{\sum_{f \in F_p} MI(f) \cdot SLOC(f)}{\sum_{f \in F_p} SLOC(f)} \quad (2)$$

dove  $B_p$  è l'insieme dei blocchi di codice del progetto  $p$  e  $F_p$  l'insieme dei suoi file Python con  $SLOC(f) > 0$ .

**Perché l'MI è pesato e la CC no.** La CC è già una misura per blocco, quindi la media aritmetica sui blocchi è la sua aggregazione naturale. L'MI è invece definito per file: pesarlo sullo SLOC evita che un file di poche righe con MI molto alto o molto basso domini il valore di progetto.

## Classificazione e tipologia di manutenzione

Tabella 13: CR1 — Classificazione

| Classificazione   | Enhancement                                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Additive</b>   | Aggiunge una nuova feature (calcolo e salvataggio di CC/MI) al tool.                                                                |
| <b>Perfective</b> | Arricchisce l'output con informazioni aggiuntive (CC e MI) che rendono l'analisi più completa, interpretabile e utile per l'utente. |

### 4.3 CR2 — GUI per configurazione ed esecuzione analisi

#### Motivazione

L'uso del tool nella situazione attuale richiede interventi manuali su parametri *hard-coded* nello script `main.py` (limite L2 di tabella 2), rendendo meno pratico eseguire velocemente più run con configurazioni diverse e limitando l'accesso ad utenti non tecnici.

#### Situazione desiderata (To-Be)

Introdurre una GUI (Tkinter) che consenta di:

- selezionare facilmente gli input (directory locale oppure CSV per il cloning);
- configurare i parametri di analisi (ruolo Producer/Consumer, opzioni regole, attivazione metriche);
- avviare l'analisi tramite un pulsante;
- consultare i risultati direttamente dall'interfaccia.

#### Descrizione della modifica

La GUI organizza l'utilizzo del tool come un flusso guidato:

1. **Input selection:** scelta della sorgente (cartella locale o cloning da CSV).
2. **Configuration:** impostazione dei parametri principali (ruolo, regole, numero di repo se cloning, abilitazione metriche).
3. **Execution:** avvio della run **senza bloccare l'interfaccia**, tramite gestione asincrona su thread separato.
4. **Results:** visualizzazione dei file prodotti e dei contenuti principali (tabella o viewer integrato).

## Classificazione e tipologia di manutenzione

Tabella 14: CR2 — Classificazione

| Classificazione | Enhancement                                                                                                                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Additive        | Aggiunge una nuova feature (GUI) per configurare ed eseguire il tool e per consultare i risultati, estendendo l'uso del sistema senza modificarne l'obiettivo funzionale principale. |

### 4.4 CR3 — Dashboard di reportistica integrata nella GUI

#### Motivazione

Nella versione attuale i risultati sono principalmente consultabili tramite file CSV (limite L3 di tabella 2). Per analisi su più repository è utile una vista aggregata che fornisca *insight* rapidi e confrontabili, evitando l'interpretazione manuale dei file.

#### Situazione desiderata (To-Be)

Aggiungere una sezione “Dashboard” nella GUI con grafici (matplotlib integrato) e statistiche aggregate per run.

#### Descrizione della modifica

Dopo l'esecuzione di una run, la GUI deve poter costruire una dashboard basata sugli output generati (CSV di risultati e metriche). La dashboard è pensata come una vista di sintesi per dataset di repository.

Tabella 15: CR3 — Viste previste nella dashboard

| Vista                         | Contenuto                                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Analysis Overview             | Numero e percentuale di progetti classificati come Producer, Consumer e Producer & Consumer.                               |
| Code Quality Metrics Overview | Riepilogo delle metriche introdotte da CR1 (CC e MI medi).                                                                 |
| Top ML Keywords               | Top 10 delle keyword ML (di training o inferenza) più rilevate per i risultati analizzati, con conteggio delle occorrenze. |

## Classificazione e tipologia di manutenzione

Tabella 16: CR3 — Classificazione

| Classificazione | Enhancement                                                                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Additive        | Aggiunge una nuova feature di reportistica.                                                                                                                                                                       |
| Perfective      | Fornisce miglioramenti informativi per gli utenti (viste aggregate, insight e visualizzazioni), migliorando la fruibilità dei risultati della run e supportando valutazioni complessive su dataset di repository. |

## 5 Master Test Plan

Definite le modifiche, prima di implementarle è stato fissato l'approccio di testing con cui verificarle. Il presente **Master Test Plan** vale per MARK 2.0 Plus nel suo complesso — funzionalità della baseline ed evoluzioni introdotte dalle tre Change Request — e definisce:

- i livelli di test previsti (unità, integrazione, sistema);
- le tecniche utilizzate;
- i componenti su cui concentrare lo sforzo di verifica;
- un criterio di copertura strutturale per le parti testate in white-box.

### 5.1 Strategia di testing

La strategia è articolata su tre livelli, riassunti in tabella 17.

**Tabella 17:** Livelli di test, tecniche e criteri di selezione

| Livello             | Tecnica   | Selezione input       | Dipendenze                   |
|---------------------|-----------|-----------------------|------------------------------|
| Basic Unit Testing  | White-box | CFG dei metodi target | Mockate dove necessario      |
| Integration Testing | White-box | CFG dei metodi target | Reali (nessun mock)          |
| System Testing      | Black-box | Category Partition    | Sistema completo via CLI/GUI |

#### 5.1.1 Basic Unit Testing (white-box)

Verrà effettuato un **basic unit testing** focalizzato sui **metodi** delle classi selezionate, con l'obiettivo di verificare la logica interna dei rami principali. Quando necessario, le dipendenze verranno **mockate** per isolare il metodo sotto test e controllare lo stato dell'ambiente (ad es. filesystem, servizi del tool).

La progettazione dei test avverrà in modalità **white-box**: per ciascun metodo target verrà costruito il relativo **Control Flow Graph (CFG)** tramite uno script Python basato su **AST**. A partire dal CFG verranno individuati i percorsi che coprono il maggior numero di branch e verranno scelti gli input in grado di attivarli. I percorsi selezionati verranno poi implementati come casi di test.

#### 5.1.2 Integration Testing (white-box)

Verrà eseguito anche **integration testing** sui componenti scelti, per verificare le interazioni tra classi e servizi senza isolare artificialmente il comportamento. In questa fase **non verranno mockate le dipendenze**: i test utilizzeranno interazioni reali (filesystem reale o temporaneo, servizi del tool).

Anche qui la selezione dei test verrà guidata dal CFG, con l'obiettivo di coprire i branch più significativi dei metodi di interesse.

#### 5.1.3 System Testing (black-box, Category Partition)

Il **system testing** verrà condotto in modalità **black-box** progettando i casi di test tramite **Category Partition** [4], con il procedimento già descritto in sezione 3.2: la definizione delle categorie e delle scelte riguarderà i parametri di input e lo stato dell'ambiente, che insieme agli oracoli saranno ricavati dalle specifiche delle funzionalità del sistema espresse in forma di casi d'uso.

I test di sistema saranno pianificati:

- sul **sistema pre-modifica**, così da ottenere una suite riutilizzabile per **regressione** post-CR (sezione 3);
- sulle funzionalità introdotte da **CR1–CR3**, con casi dedicati per ciascuna nuova feature (sezione 7.2).

Per favorire ripetibilità ed evitare modifiche manuali a parametri *hard-coded*, dove opportuno l'esecuzione dei test di sistema verrà effettuata tramite `main_args.py` con pochi flag essenziali e valori di default invariati (ad es. analisi su directory o cloning da CSV).

## 5.2 Componenti incluse, componenti escluse e coverage

### 5.2.1 Componenti incluse

**Unit testing** (basic, sui metodi):

- tutti i metodi del sistema che saranno **modificati dalla CR1**;
- metodi delle classi **Service** della GUI, responsabili dell'interfacciamento tra Controller e servizi del sistema o filesystem.

**Integration testing:**

- gli stessi metodi scelti per l'unit testing;
- metodi della classe **Controller** della GUI, per verificare la logica di orchestrazione tra componenti grafiche e classi Service.

**Tabella 18:** Componenti target del testing white-box

| Componente                       | Unit | Integration | CR      |
|----------------------------------|------|-------------|---------|
| modules/analyzer/ml_analyzer.py  | ✓    | ✓           | CR1     |
| gui/services/pipeline_service.py | ✓    | ✓           | CR2     |
| gui/services/output_reader.py    | ✓    | ✓           | CR2/CR3 |
| gui/controller.py                | —    | ✓           | CR2/CR3 |

### 5.2.2 Componenti escluse

Le classi **View** della GUI verranno escluse dal testing white-box, poiché si occupano prevalentemente di **logica di presentazione** (layout, rendering) e non contengono logica applicativa core; lo sforzo di verifica viene quindi concentrato su Controller, Service e sui componenti di analisi modificati.

Le View restano comunque verificate **indirettamente** a livello di sistema: i test black-box della CR2 e della CR3 pilotano l'interfaccia reale e ne osservano gli effetti, quindi un difetto di presentazione che alteri il comportamento osservabile viene rilevato.

### 5.2.3 Coverage

Per unit e integration testing verrà adottato un obiettivo di  $\geq 80\%$  di **branch coverage** sulle classi di interesse del testing. La copertura verrà misurata con **pytest-cov** [7] e calcolata **solo** sul sottoinsieme di componenti target elencato in tabella 18.

```

1 python -m pytest test/unit_testing \
2   --cov=modules.analyzer.ml_analyzer \
3   --cov=gui.services \
4   --cov-branch --cov-report=term --cov-report=html

```

**Listing 2:** Comando di misura della branch coverage

## 6 Impact Analysis

Prima di implementare le Change Request è stato stimato il loro impatto sui componenti esistenti. La sezione riporta, per ciascuna CR, il metodo adottato, gli insiemi di impatto previsti e — a valle dell’implementazione — il confronto con l’impatto effettivamente osservato.

### 6.1 Metodo adottato e scelta degli SLO

In assenza di artefatti di requisiti e design (ad es. SRS, diagrammi UML aggiornati), l’Impact Analysis viene condotta sugli artefatti disponibili — il codice sorgente — usando *traceability* interna (vertical traceability) e relazioni di dipendenza tra componenti.

Il riferimento teorico utilizzato [1] introduce gli **SLO (Software Life-cycle Objects)** come oggetti che possono variare di granularità. Nel nostro contesto, per mantenere una granularità uniforme e compatibile con un’analisi call-graph-based in ambiente OO, si assumono come SLO i **metodi** del sistema software.

Le due CR che introducono l’interfaccia grafica sono state progettate in modo da non toccare il core, e per questo richiedono un’analisi diversa da quella della CR1: le due situazioni sono trattate separatamente nelle sezioni 6.2 e 6.3.

### 6.2 CR1 — Metriche di qualità del codice

La CR1 introduce un’estensione funzionale al core: integrare il calcolo di metriche di qualità del codice (Radon [8]), in particolare **Cyclomatic Complexity (CC)** [2] e **Maintainability Index (MI)** [3], calcolate sui file sorgente analizzati e aggregate a livello di progetto secondo equazioni (1) e (2), con salvataggio dei risultati in output allineato alla run corrente.

#### 6.2.1 Starting Impact Set (SIS)

Sulla base dell’analisi del sistema (sezione 2) e del contenuto della CR1, vengono identificati come *core methods* da modificare i tre metodi “template” della classe `MLAnalyzer`, responsabili del workflow principale di analisi:

$$\text{SIS} = \left\{ \begin{array}{l} \text{MLAnalyzer.analyze\_single\_file,} \\ \text{MLAnalyzer.analyze\_project,} \\ \text{MLAnalyzer.analyze\_projects\_set} \end{array} \right\} \quad |\text{SIS}| = 3$$

#### 6.2.2 Candidate Impact Set (CIS): costruzione del call graph

La modifica dei metodi core di `MLAnalyzer` può propagarsi:

- verso i **callee** (fan-out): nuove dipendenze invocate per calcolare CC/MI e salvarle;
- verso i **caller** (fan-in): eventuali adattamenti dovuti a variazioni di ritorni o dati prodotti, o a step aggiunti alla pipeline.

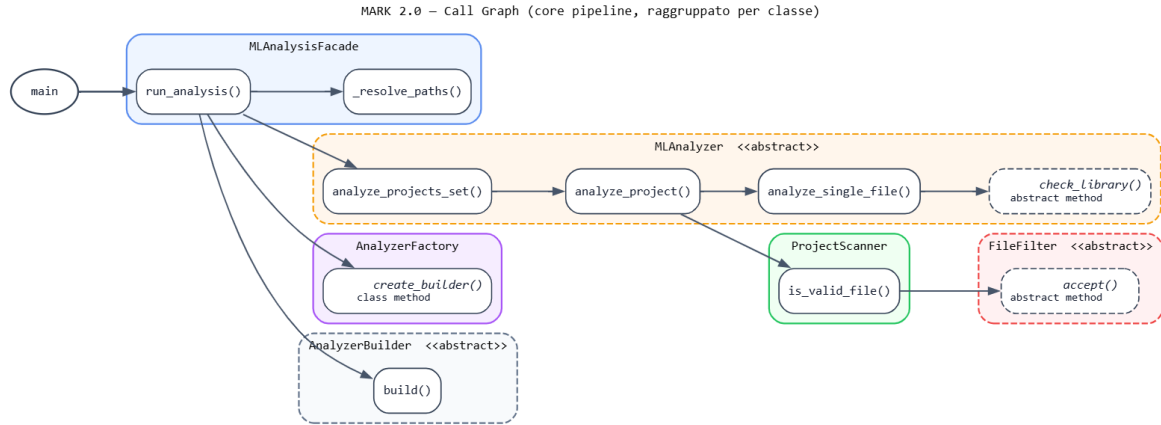
Per questo motivo si costruisce un sottografo includendo sia fan-out sia fan-in, così da ottenere una stima **prudenziale** dei candidati al cambiamento. La procedura adottata è:

1. **Seeding**: inizializzazione dei nodi del grafo con gli SLO del SIS.
2. **Espansione fan-out**: aggiunta iterativa dei metodi chiamati (callee) dai nodi correnti.
3. **Espansione fan-in**: aggiunta iterativa dei metodi chiamanti (caller) che invocano i nodi correnti.
4. **Arresto**: terminazione quando non emergono nuovi nodi interni al sistema.



5. **Filtraggio**: esclusione di nodi esterni (librerie di terze parti o built-in) per mantenere il grafo sugli SLO del sistema.

Il risultato è un sottografo statico che rappresenta una **sovra-approssimazione** dei possibili impatti.



**Figura 13:** Call graph della core pipeline di MARK 2.0, raggruppato per classe

A partire dai tre metodi del SIS, il sottografo ricostruito include:

- l'entry-point della pipeline (**main**), che invoca il Facade;
- **MLAnalysisFacade.run\_analysis**, che orchestra l'analisi e attiva factory e builder;
- i metodi di costruzione (**AnalyzerFactory.create\_builder**, **AnalyzerBuilder.build**);
- i metodi di workflow dell'analyzer (**analyze\_projects\_set** → **analyze\_project** → **analyze\_single\_file**);
- i metodi ausiliari chiamati nel workflow (validazione file e filtraggio; metodo astratto **check\_library**).

### 6.2.3 Valutazione incrementale dei nodi

Il call graph costruito produce un insieme di nodi raggiungibili dai metodi del SIS e/o che li raggiungono: tale insieme è un **superset** di possibili impatti. Tuttavia, la CR1 è specifica (aggiunta di CC/MI e relativo salvataggio e aggregazione), quindi non tutti i nodi raggiungibili sono necessariamente da modificare.

Per rendere l'Impact Analysis aderente alla definizione di CIS — “insieme potenzialmente affetto” ma non arbitrariamente esteso — si introduce un passo incrementale:

1. per ciascun nodo del sottografo si valuta se la CR può richiedere modifica **diretta** del comportamento, dei dati prodotti o consumati, o dell'orchestrazione in cui il nodo è coinvolto;
2. se la pertinenza è nulla o trascurabile, il nodo viene escluso dal CIS.

La valutazione dei componenti emersi è riportata in tabella 19.

**Tabella 19:** Valutazione incrementale dei nodi del call graph

| Nodo               | Esito | Motivazione                                                                    |
|--------------------|-------|--------------------------------------------------------------------------------|
| main (entry-point) | IN    | Potrebbe essere necessario per introdurre un nuovo flag o step nella pipeline. |

| Nodo                                                                       | Esito      | Motivazione                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MLAnalysisFacade.run_analysis</code>                                 | <b>IN</b>  | L'introduzione di un nuovo ruolo di analisi può richiedere gestione specifica di output/directory e corretta attivazione della build.                                                                                                                                               |
| <code>MLAnalysisFacade._resolve_paths</code>                               | <b>OUT</b> | Gestisce la risoluzione dei path dei dizionari ML e la creazione della cartella di output principale; non è necessaria per CR1.                                                                                                                                                     |
| <code>AnalyzerFactory.create_builder</code>                                | <b>IN</b>  | Candidato prudenziale: può risultare necessario per introdurre o risolvere il nuovo ruolo <b>METRICS</b> (potrà diventare FPIS se l'estensione avviene senza modifiche alla factory).                                                                                               |
| <code>AnalyzerBuilder.build</code> (astratto)                              | <b>IN</b>  | Candidato prudenziale nel caso siano necessarie dipendenze o configurazioni diverse per la costruzione del nuovo analyzer.                                                                                                                                                          |
| <code>ProjectScanner.is_valid_file</code> / <code>FileFilter.accept</code> | <b>OUT</b> | CR1 non modifica i criteri di selezione dei file; si riusano invariati i file già analizzati.                                                                                                                                                                                       |
| <code>MLAnalyzer.check_library</code> (astratto)                           | <b>OUT</b> | La logica librerie/keyword non viene modificata; tuttavia CR1 richiede una nuova classe concreta che implementi <code>check_library</code> per il ruolo metrics (ad es. restituendo liste vuote). L'impatto è quindi sulla nuova implementazione concreta, non sul metodo astratto. |
| <code>AnalyzerRole</code> (Enum)                                           | <b>IN</b>  | Necessario per introdurre il nuovo ruolo <b>METRICS</b> .                                                                                                                                                                                                                           |

## Componenti da aggiungere

Si prevede l'introduzione dei seguenti elementi:

- `MLMetricsAnalyzer`: classe concreta di `MLAnalyzer` che esegue l'override di `check_library` restituendo liste vuote, in quanto non interessa l'estrazione di librerie o keyword;
- `MLAnalyzer._save_metrics_csv`: metodo per salvare i CSV relativi alle metriche;
- `MetricsAnalyzerBuilder`: classe concreta di `AnalyzerBuilder`, costruttore di `MLMetricsAnalyzer` richiamato dal Facade.

## File e cartelle di output impattate

- Cartella `logs`: i file `.log` potrebbero essere arricchiti con nuove informazioni di log generate dal nuovo step introdotto.
- Cartella `output`: potrà essere modificata dalla creazione di una nuova cartella risultati `metrics`, con una sottocartella per ogni run (`metrics_<num_run>`) e file `.csv` contenenti le metriche calcolate.

### 6.2.4 CIS finale

Sulla base del sottografo e della valutazione incrementale, il Candidate Impact Set viene definito come:

$$CIS_{\text{methods}} = \left\{ \begin{array}{l} \text{main, MLAnalyzer.analyze\_single\_file,} \\ \text{MLAnalyzer.analyze\_project, MLAnalyzer.analyze\_projects\_set,} \\ \text{MLAnalysisFacade.run\_analysis, AnalyzerFactory.create\_builder,} \\ \text{AnalyzerBuilder.build, MLAnalyzer.\_save\_metrics\_csv (TO\_ADD)} \end{array} \right\}$$

$$CIS_{\text{classes}} = \{ \text{AnalyzerRole, MLMetricsAnalyzer (TO\_ADD),} \\ \text{MetricsAnalyzerBuilder (TO\_ADD)} \}$$

$$CIS_{\text{folders}} = \{ \text{logs, output} \}$$

$$|CIS| = 8 + 3 + 2 = \mathbf{13}$$

### 6.2.5 Impatto effettivo dopo l'implementazione

A seguito dell'implementazione di CR1, è stato possibile verificare l'impatto effettivo delle modifiche introdotte sui componenti del sistema.

#### Actual Impact Set (AIS)

Risultano effettivamente modificati o aggiunti:

$$AIS_{\text{methods}} = \left\{ \begin{array}{l} \text{main, MLAnalyzer.analyze\_single\_file,} \\ \text{MLAnalyzer.analyze\_project, MLAnalyzer.analyze\_projects\_set,} \\ \text{MLAnalyzer.\_save\_metrics\_csv (ADDED)} \end{array} \right\}$$

$$AIS_{\text{classes}} = \{ \text{AnalyzerRole, MLMetricsAnalyzer (ADDED),} \\ \text{MetricsAnalyzerBuilder (ADDED)} \}$$

$$AIS_{\text{folders}} = \{ \text{logs, output} \}$$

$$|AIS| = 5 + 3 + 2 = \mathbf{10}$$

#### False Positive Impact Set (FPIS)

Sono risultati falsi positivi i seguenti elementi previsti nel CIS ma non modificati nell'implementazione:

$$FPIS = CIS \setminus AIS = \left\{ \begin{array}{l} \text{MLAnalysisFacade.run\_analysis,} \\ \text{AnalyzerFactory.create\_builder,} \\ \text{AnalyzerBuilder.build} \end{array} \right\} \quad |FPIS| = 3$$

Il motivo è che il pattern **Factory-Registry** si è rivelato sufficientemente estensibile: la registrazione del nuovo builder tramite decoratore non ha richiesto alcuna modifica né alla factory né al Facade.

#### Discovered Impact Set (DIS)

Non sono stati individuati componenti impattati non inclusi nel CIS:

$$DIS = AIS \setminus CIS = \emptyset$$

### 6.2.6 Metriche di qualità dell'analisi

Per valutare la qualità dell'Impact Analysis si utilizzano le metriche tipiche di information retrieval applicate ai set CIS/AIS:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{10}{13} \approx 0,77 \quad (3)$$

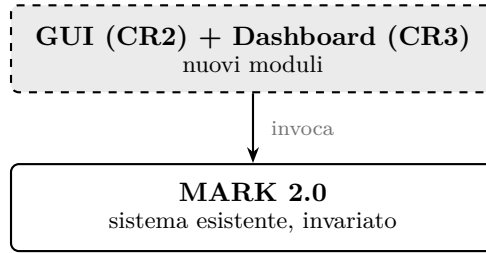
$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{10}{10} = 1 \quad (4)$$

In sintesi, l'analisi è stata **conservativa**: ha massimizzato la copertura dei componenti realmente impattati (recall ottimo), al costo di una moderata sovra-approssimazione ( $Precision < 1$ ). In un contesto di manutenzione questo è il compromesso desiderabile: un falso positivo costa una verifica in più, un falso negativo costa un difetto non individuato.

### 6.3 CR2 (GUI) e CR3 (Dashboard)

Per le CR2 e CR3 si adotta un approccio differente rispetto alla CR1.

Entrambe le modifiche sono state progettate **come moduli separati**, con l'obiettivo esplicito di **non modificare il codice esistente** del core (analisi/cloning). In particolare, GUI e Dashboard operano come *strati aggiuntivi* che **invocano** funzionalità già presenti, interfacciandosi al core come “client”, senza richiedere refactoring o variazioni a classi o metodi preesistenti.



**Figura 14:** CR2 e CR3 come strati aggiuntivi sopra il core invariato

Di conseguenza, l'Impact Analysis sui componenti già presenti nel sistema risulta **banalmente nulla**: non è prevista propagazione dell'impatto sul core e non è necessario costruire un call graph basato sui fan-in/fan-out come in CR1. Dato che le CR sono implementate **solo tramite componenti aggiunti** — un nuovo sottosistema GUI/Dashboard — e con il vincolo progettuale di non modifica del core, i set relativi ai componenti esistenti sono:

$$SIS = \emptyset \quad CIS = \emptyset \quad AIS = \emptyset \quad FPIS = \emptyset \quad DIS = \emptyset$$

Le metriche di equazioni (3) e (4) applicate ai soli componenti preesistenti non risultano quindi informative (rapporti 0/0). L'analisi è volutamente **semplificata** e si limita a esplicitare che l'impatto delle nuove componenti non comporta propagazione sulle componenti esistenti.

**Verifica a posteriori.** Il vincolo “no changes to core” è stato confermato empiricamente: la suite di 16 test di sistema pre-modifica è stata rieseguita dopo CR2 e CR3 senza alcun fallimento (sezione 7.3), e il diff sul core non mostra modifiche imputabili a queste due CR.

## 6.4 Riepilogo

**Tabella 20:** Riepilogo dell'Impact Analysis per le tre Change Request

| CR  | SIS | CIS | AIS | FPIS | DIS | Metriche                       | Impatto |
|-----|-----|-----|-----|------|-----|--------------------------------|---------|
| CR1 | 3   | 13  | 10  | 3    | 0   | Precision = 0,77<br>Recall = 1 | Alto    |
| CR2 | 0   | 0   | 0   | 0    | 0   | Non informative                | Basso   |
| CR3 | 0   | 0   | 0   | 0    | 0   | Non informative                | Basso   |

## 7 Testing post-modifica

Implementate le tre Change Request, il sistema è stato verificato applicando il Master Test Plan della sezione 5 su tre fronti:

- **Unit e Integration Testing** (sezione 7.1): verifica **white-box** della correttezza dei metodi ritenuti critici nelle componenti modificate o aggiunte, con obiettivo di  $\geq 80\%$  branch coverage sulle classi target;
- **Post-Modification System Testing** (sezione 7.2): validazione a livello di sistema delle nuove funzionalità introdotte dalle CR, tramite testing **black-box** progettato con Category Partition;
- **Regression Testing** (sezione 7.3): verifica che il sistema preesistente non abbia subito regressioni funzionali, tramite la riesecuzione della suite di 16 test di sistema definita in sezione 3.

### 7.1 Unit e Integration Testing (white-box)

Per le componenti selezionate in tabella 18 è stato svolto:

- **Basic Unit Testing (white-box)**: focalizzato sui metodi, con mocking dove necessario per isolare dipendenze (filesystem, servizi);
- **Integration Testing (white-box)**: sugli stessi metodi ma senza mocking, verificando la corretta interazione con il sistema.

La selezione degli input è stata guidata dalla costruzione del CFG dei metodi target, secondo il criterio fissato in sezione 5.1.1: costruzione del grafo tramite script Python basato su AST e scelta dei percorsi che coprono il maggior numero di branch.

#### 7.1.1 Basic Unit Testing — casi di test

**Tabella 21:** Casi di test unitari

| Test ID   | Classe/Metodo                  | Path target                                                                     |
|-----------|--------------------------------|---------------------------------------------------------------------------------|
| UT-CR1-01 | MLAnalyzer.analyze_single_file | Il file non esiste.                                                             |
| UT-CR1-02 | MLAnalyzer.analyze_single_file | Errore nella lettura del file.                                                  |
| UT-CR1-03 | MLAnalyzer.analyze_single_file | Il file viene letto con successo, CC e MI sollevano eccezioni, keyword trovate. |

| Test ID   | Classe/Metodo                                    | Path target                                                                                                         |
|-----------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| UT-CR1-04 | <code>MLAnalyzer.analyze_single_file</code>      | CC e MI hanno successo, ma non vengono trovate keyword.                                                             |
| UT-CR1-05 | <code>MLAnalyzer.analyze_project</code>          | Role $\neq$ METRICS: include file non valido, file valido senza keyword, file valido con keyword.                   |
| UT-CR1-06 | <code>MLAnalyzer.analyze_project</code>          | Role = METRICS: include file con SLOC $> 0$ e file con SLOC = 0.                                                    |
| UT-CR1-07 | <code>MLAnalyzer.analyze_projects_set</code>     | Role $\neq$ METRICS con progetto non-directory, percorso non-directory e directory valida che ritorna df non vuoto. |
| UT-CR1-08 | <code>MLAnalyzer.analyze_projects_set</code>     | Role = METRICS con progetto A (cc/sloc vuoti) e progetto B (con cc/sloc), tutti i df vuoti.                         |
| UT-CR2-01 | <code>PipelineService.run_pipeline</code>        | Cloning + cloner check abilitati, analisi disabilitate.                                                             |
| UT-CR2-02 | <code>PipelineService.run_pipeline</code>        | Tutte le analisi abilitate (producer, consumer, metrics), nessun cloning.                                           |
| UT-CR2-03 | <code>PipelineService.run_pipeline</code>        | Percorso CSV non valido: deve gestire l'errore correttamente.                                                       |
| UT-CR2-04 | <code>OutputReader.scan_output_tree</code>       | Directory di output vuota $\rightarrow$ restituisce albero vuoto.                                                   |
| UT-CR2-05 | <code>OutputReader.scan_output_tree</code>       | Tutte le categorie (producer, consumer, metrics) con file CSV.                                                      |
| UT-CR2-06 | <code>OutputReader.load_csv</code>               | File non esistente $\rightarrow$ solleva <code>FileNotFoundException</code> .                                       |
| UT-CR2-07 | <code>OutputReader.load_csv</code>               | File CSV valido $\rightarrow$ restituisce <code>CSVData</code> con headers e righe.                                 |
| UT-CR2-08 | <code>OutputReader.load_csv</code>               | File CSV vuoto $\rightarrow$ restituisce <code>CSVData</code> vuoto.                                                |
| UT-CR3-01 | <code>OutputReader.find_complete_analyses</code> | Nessuna directory di analisi $\rightarrow$ restituisce lista vuota.                                                 |
| UT-CR3-02 | <code>OutputReader.find_complete_analyses</code> | Tutte le categorie con stesso ID analisi $\rightarrow$ restituisce quell'ID.                                        |

### 7.1.2 Integration Testing — casi di test

Gli stessi path coperti dai test case da UT-CR1-01 a UT-CR3-02 sono stati riutilizzati per generare un numero equivalente di test case di integrazione. Poiché tali path risultano identici e ripetitivi, se ne omette la descrizione dettagliata.

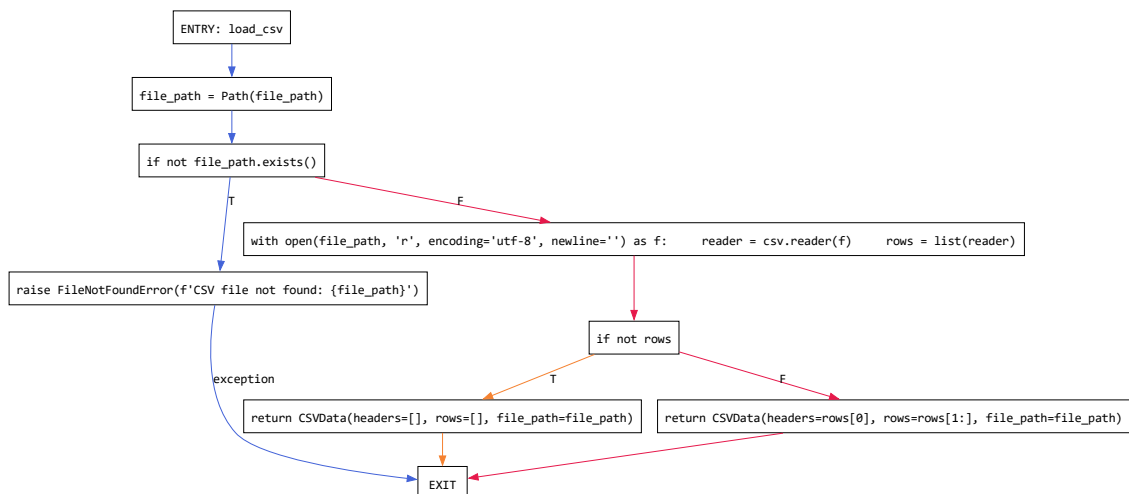
In aggiunta, sono stati testati i metodi della classe `AppController` appartenente alla GUI.

**Tabella 22:** Casi di test di integrazione specifici del Controller

| Test ID   | Classe/Metodo                       | Path target                                                         |
|-----------|-------------------------------------|---------------------------------------------------------------------|
| IT-CR2-09 | AppController._on_start_pipeline    | Repo path non esiste → messaggio d'errore.                          |
| IT-CR2-10 | AppController._on_start_pipeline    | Repo path esiste → pipeline avviata con thread.                     |
| IT-CR2-11 | AppController._on_pipeline_complete | Pipeline completata con successo → mostra info e aggiorna output.   |
| IT-CR2-12 | AppController._on_pipeline_complete | Pipeline fallisce → mostra errore.                                  |
| IT-CR2-13 | AppController._on_pipeline_complete | Pipeline completa senza result → errore sconosciuto.                |
| IT-CR3-03 | AppController._refresh_output_tree  | Aggiorna tree con successo quando esistono analisi.                 |
| IT-CR3-04 | AppController._on_file_select       | Selezione file CSV valido → mostra dati.                            |
| IT-CR3-05 | AppController._on_file_select       | File non esiste → mostra errore.                                    |
| IT-CR3-06 | AppController._on_analysis_select   | Producer/Consumer/Metrics CSV esistono → calcola metriche complete. |
| IT-CR3-07 | AppController._on_analysis_select   | Producer/Consumer CSV non trovati → metriche a zero.                |

### 7.1.3 Esempio di CFG e tracciamento dei path

Metodo selezionato per il basic unit testing: `OutputReader.load_csv`.



**Figura 15:** CFG generato per `OutputReader.load_csv`

**Tabella 23:** Path coperti sul CFG di figura 15

| Path | Test case | Comportamento                                                       |
|------|-----------|---------------------------------------------------------------------|
| P1   | UT-CR2-06 | File non esistente → solleva <code>FileNotFoundError</code> .       |
| P2   | UT-CR2-07 | File CSV valido → ritorna <code>CSVData</code> con headers e righe. |
| P3   | UT-CR2-08 | File CSV vuoto → ritorna <code>CSVData</code> vuoto.                |

Tutti i CFG utilizzati nella progettazione white-box sono versionati come sorgenti Graphviz sotto `test/white_box_paths/` e sono rigenerabili in formato vettoriale con `python tools/render_cfg.py`.

#### 7.1.4 Coverage (pytest-cov, branch coverage)

La coverage è stata calcolata con **pytest-cov** [7] limitatamente alle **classi di interesse** — quelle effettivamente testate in white-box — misurando in particolare la **branch coverage** con soglia obiettivo  $\geq 80\%$ .

**Tabella 24:** Branch coverage sulle classi target, per livello di test

| Package/Classe                   | Unit                | Integration         | CR      |
|----------------------------------|---------------------|---------------------|---------|
| modules/analyzer/ml_analyzer.py  | 98 % (43/44)        | 100 % (44/44)       | CR1     |
| gui/services/pipeline_service.py | 100 % (10/10)       | 100 % (10/10)       | CR2     |
| gui/services/output_reader.py    | 93 % (13/14)        | 93 % (13/14)        | CR2/CR3 |
| gui/controller.py                | —                   | 68 % (19/28)        | CR2/CR3 |
| <b>Totale</b>                    | <b>97 % (66/68)</b> | <b>90 % (86/96)</b> |         |

Lo Unit Testing supera l'obiettivo del Master Test Plan ( $\geq 80\%$ ) su tutte le classi target. Nell'Integration Testing lo superano tre classi su quattro: `gui/controller.py` si ferma al 68 %. I valori sono riproducibili con `pytest --cov` e i report HTML completi generati da **coverage.py** sono versionati sotto `docs/coverage/`.

## 7.2 Post-Modification System Testing

Il system testing è condotto in modalità **black-box**, progettando i casi di test tramite **Category Partition** secondo il procedimento descritto in sezione 3.2. Sono stati definiti tre casi d'uso dedicati alle nuove funzionalità: UC-CR1 (metriche), UC-CR2 (GUI) e UC-CR3 (dashboard).



### 7.2.1 UC-CR1 — Analisi del calcolo delle metriche

**Tabella 25:** UC-CR1 — Calcolo delle metriche

|                                    |                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrizione</b>                 | L'utente esegue l'analisi su una directory locale contenente repository già disponibili; il tool calcola le metriche Maintainability Index (MI) e Cyclomatic Complexity (CC) per i progetti contenuti, salvando i risultati in <code>io/output/</code> .                                                                                       |
| <b>Attori</b>                      | Utente (ricercatore)                                                                                                                                                                                                                                                                                                                           |
| <b>Entry condition</b>             | <code>main_args.py</code> disponibile; dipendenze installate (inclusa Radon); directory <code>--repository-path</code> accessibile (per scenari "success").                                                                                                                                                                                    |
| <b>Exit condition</b>              | <i>Successo:</i> metriche (MI, CC) calcolate e risultati salvati in <code>io/output/</code> .<br><i>Errore:</i> messaggio d'errore e assenza di output significativo.                                                                                                                                                                          |
| <b>Flusso di eventi principale</b> | <ol style="list-style-type: none"> <li>1. Invocazione con <code>--repository-path</code> e <code>--metrics</code>.</li> <li>2. Validazione del path.</li> <li>3. Scansione dei repository nella directory.</li> <li>4. Calcolo MI e CC sui progetti analizzati.</li> <li>5. Salvataggio output metriche in <code>io/output/</code>.</li> </ol> |

**Parametro:** path della directory contenente i repository (`repository_path`). **Oggetti dell'ambiente:** filesystem, contenuto dei progetti.

**Tabella 26:** UC-CR1 — Categorie e scelte

| Categoria                 | ID  | Descrizione                                          | Proprietà / Vincoli                |
|---------------------------|-----|------------------------------------------------------|------------------------------------|
| Esistenza input directory | ED1 | Directory non esiste                                 | [Error]                            |
|                           | ED2 | Directory esiste                                     | [property DirOk]                   |
| Contenuto directory       | CD0 | Directory vuota (0 progetti)                         | [if DirOk] [property Empty]        |
|                           | CD1 | Directory con 1+ progetti                            | [if DirOk] [property MultiProject] |
| Composizione progetti     | CP0 | Tutti i progetti senza file Python                   | [if DirOk and MultiProject]        |
|                           | CP1 | Tutti i progetti con file Python ma vuoti            | [if DirOk and MultiProject]        |
|                           | CP2 | Tutti i progetti con file Python validi e con codice | [if DirOk and MultiProject]        |
|                           | CP3 | Mix di progetti validi e non validi                  | [if DirOk and MultiProject]        |

**Tabella 27:** UC-CR1 — Test frame

| ID  | Combinazioni  | Oracolo (risultato atteso)                      |
|-----|---------------|-------------------------------------------------|
| TF1 | ED1           | Messaggio di errore "Input folder not found"    |
| TF2 | ED2, CD0      | Nessuna metrica calcolata                       |
| TF3 | ED2, CD1, CP0 | Per tutti i progetti MI e CC pari a 0           |
| TF4 | ED2, CD1, CP1 | Per tutti i progetti MI e CC pari a 0           |
| TF5 | ED2, CD1, CP2 | Per tutti i progetti MI e CC sono valori esatti |
| TF6 | ED2, CD1, CP3 | Per tutti i progetti MI e CC sono valori esatti |

**Tabella 28:** UC-CR1 — Test case

| TC  | Input<br>(repository_<br>path) | Descrizione<br>ambiente                                                    | Oracolo                                                                                                                                                                                                                                                                                                                  |
|-----|--------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC1 | repo_does_not_exist            | Directory inesistente                                                      | Messaggio “Input folder not found”                                                                                                                                                                                                                                                                                       |
| TC2 | empty_repo                     | Directory vuota                                                            | Nessuna metrica calcolata                                                                                                                                                                                                                                                                                                |
| TC3 | test_repos/TC3                 | Più progetti senza file Python                                             | Per tutti i progetti MI e CC pari a 0                                                                                                                                                                                                                                                                                    |
| TC4 | test_repos/TC4                 | Più progetti con file Python vuoti                                         | Per tutti i progetti MI e CC pari a 0                                                                                                                                                                                                                                                                                    |
| TC5 | test_repos/TC5                 | Più progetti con file Python contenenti codice valido                      | Valori esatti calcolati manualmente:<br>project1: CC_avg = 1,67, MI_avg = 77,5<br>project2: CC_avg = 1,33, MI_avg = 88,75                                                                                                                                                                                                |
| TC6 | test_repos/TC6                 | Mix: progetti senza file Python, con file Python vuoti e con codice valido | Valori esatti per ogni progetto:<br>project_empty_python_1: CC_avg = 0, MI_avg = 0<br>project_empty_python_2: CC_avg = 0, MI_avg = 0<br>project_no_python_1: CC_avg = 0, MI_avg = 0<br>project_no_python_2: CC_avg = 0, MI_avg = 0<br>project1: CC_avg = 1,67, MI_avg = 77,51<br>project2: CC_avg = 1,33, MI_avg = 88,75 |

## 7.2.2 UC-CR2 — Configurazione e avvio analisi tramite GUI

**Tabella 29:** UC-CR2 — Configurazione ed esecuzione da GUI

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrizione</b>                 | L'utente configura i parametri della pipeline nella GUI e avvia l'analisi; al termine il sistema salva i risultati in <code>io/output/</code> e mostra l'esito (successo/errore).                                                                                                                                                                                                                                                                                                   |
| <b>Attori</b>                      | Utente (ricercatore)                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Entry condition</b>             | GUI aperta; utente nella tab Configuration; interfaccia in stato Idle (pulsante Start Analysis abilitato).                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Exit condition</b>              | <i>Successo:</i> pipeline completata, risultati salvati in <code>io/output/</code> , dialogo di successo mostrato, interfaccia in stato Idle sulla tab Output.<br><i>Errore:</i> pipeline fallita o non avviata, dialogo di errore/validazione mostrato, interfaccia in stato Idle sulla tab Configuration.                                                                                                                                                                         |
| <b>Flusso di eventi principale</b> | <ol style="list-style-type: none"> <li>1. L'utente imposta i path (IO Path, Repository Path, Project List Path), il numero di repo da clonare, seleziona i pipeline step e l'opzione Enable Rules</li> <li>2. L'utente avvia l'analisi con Start Analysis.</li> <li>3. Il sistema valida i parametri, disabilita Start Analysis e avvia la pipeline in un thread separato.</li> <li>4. Il sistema completa l'analisi, salva i risultati e mostra un dialogo di successo.</li> </ol> |

### Parametri:

- `io_path`, `repos_path`, `project_list_path` (String);
- `num_repos` (Integer, 1–1000);
- `clone_enabled`, `verify_enabled`, `producer_enabled`, `consumer_enabled`, `metrics_enabled`, `rules3_enabled` (Boolean).

**Oggetti dell'ambiente:** stato del filesystem; contenuto e validità del file CSV dei progetti; stato della cartella contenente i repository.

**Tabella 30:** UC-CR2 — Categorie e scelte

| Categoria                | ID  | Descrizione                      | Vincoli                        |
|--------------------------|-----|----------------------------------|--------------------------------|
| Step selezionati         | ST0 | Nessuno step selezionato         | [Single]                       |
|                          | ST1 | Almeno uno step selezionato      | [property PARTIAL]             |
|                          | ST2 | Tutti gli step selezionati       | [property ALL] [Single]        |
| Cloning + Verify         | CV1 | Cloning e Verify selezionati     | [if PARTIAL] [property CV_ON]  |
|                          | CV2 | Cloning e Verify non selezionati | [if PARTIAL] [property CV_OFF] |
| Esistenza IO directory   | IO0 | IO directory non esiste          | [Error]                        |
|                          | IO1 | IO directory esiste              | [property IO_OK]               |
| Esistenza directory repo | RP0 | Directory repo non esiste        | [Single]                       |
|                          | RP1 | Directory repo esiste            | [property REPO_OK]             |

| <b>Categoria</b>   | <b>ID</b> | <b>Descrizione</b>                  | <b>Vincoli</b>                                      |
|--------------------|-----------|-------------------------------------|-----------------------------------------------------|
| Esistenza file CSV | CSV0      | File CSV non esiste                 | [if CV_ON and IO_OK and REPO_OK] [Error]            |
|                    | CSV1      | File CSV esiste                     | [if CV_ON and IO_OK and REPO_OK] [property CSV_OK]  |
| Stato CSV          | CS0       | File CSV vuoto                      | [if CSV_OK] [Single] [property CSV_EMPTY]           |
|                    | CS1       | File CSV non vuoto, almeno una riga | [if CSV_OK] [property CSV_NONEMPTY]                 |
| Rule 3             | RU3_0     | Regola 3 non selezionata            | [property RU3_OFF] [Single]                         |
|                    | RU3_1     | Regola 3 selezionata                | [property RU3_ON] [Single]                          |
| Valore N-repos     | N1        | N-repos < 0                         | [if CSV_NONEMPTY and CV_ON] [Error]                 |
|                    | N2        | N-repos = 0                         | [if CSV_NONEMPTY and CV_ON] [Single]                |
|                    | N3        | 0 < N-repos < #righeCSV             | [if CSV_NONEMPTY and CV_ON] [property N_OK]         |
|                    | N4        | N-repos > #righeCSV                 | [if CSV_NONEMPTY and CV_ON] [Error] [property N_GT] |

**Tabella 31:** UC-CR2 — Test frame

| <b>ID</b> | <b>Combinazioni (categorie/scelte)</b> | <b>Oracolo (risultato atteso)</b>                                |
|-----------|----------------------------------------|------------------------------------------------------------------|
| TF1       | ST0                                    | [Single] Nessuno step                                            |
| TF2       | ST1, CV1, IO0                          | [Error] IO dir non esiste                                        |
| TF3       | ST1, CV1, IO1, RP0                     | [Single] Repo dir non esiste e viene creata                      |
| TF4       | ST1, CV1, IO1, RP1, CSV0               | [Error] CSV mancante                                             |
| TF5       | ST1, CV1, IO1, RP1, CSV1, CS0, RU3_0   | [Single] Regola 3 non eseguita e analisi completata con successo |
| TF6       | ST1, CV1, IO1, RP1, CSV1, CS0, RU3_1   | [Single] Regola 3 eseguita e analisi completata con successo     |
| TF7       | ST1, CV1, IO1, RP1, CSV1, CS1, N1      | [Error] N-repos < 0                                              |
| TF8       | ST1, CV1, IO1, RP1, CSV1, CS1, N2      | [Single] N-repos = 0 e analisi completata con successo           |
| TF9       | ST1, CV1, IO1, RP1, CSV1, CS1, N3      | Analisi completata con successo                                  |
| TF10      | ST1, CV1, IO1, RP1, CSV1, CS1, N4      | [Error] N-repos > #righeCSV                                      |

| ID   | Combinazioni                      | Oracolo                                                            |
|------|-----------------------------------|--------------------------------------------------------------------|
| TF11 | ST1, CV2, IO1, RP1                | Analisi completata con successo (solo step Producer)               |
| TF12 | ST2, CV1, IO1, RP1, CSV1, CS1, N3 | [Single] Tutti gli step eseguiti e analisi completata con successo |

I dodici test frame sono stati istanziati in altrettanti test case eseguibili, implementati in `test/system_testing/gui_test/gui_test.py`. Ogni test case pilota la GUI reale impostando i parametri, avviando la pipeline e verificando lo stato dell'interfaccia e i dialoghi mostrati.

**Tabella 32:** UC-CR2 — Esempio di test case (TF11 e TF12)

| TC   | Input                                                                                                                                                                                | Descrizione ambiente                                                                                          | Oracolo                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| TC11 | Solo <i>Producer</i> selezionato.<br>Directory IO: <code>temp_io_structure</code> .<br>Directory repo: <code>repos</code> .                                                          | Directory IO esiste. Directory repo esiste.                                                                   | Solo lo step <i>Producer</i> abilitato.<br>Titolo: 'Success'; messaggio: 'Pipeline completed successfully!'. |
| TC12 | Tutti gli step selezionati.<br>Directory IO: <code>temp_io_structure</code> .<br>Directory repo: <code>repos</code> .<br>File CSV: <code>projects_TF12.csv</code> . Numero repos: 3. | Directory IO e repo esistono. Il CSV contiene <code>owner1/project1</code> ... <code>owner5/project5</code> . | Tutti gli step eseguiti.<br>Titolo: 'Success'; messaggio: 'Pipeline completed successfully!'.                |

**Nota sull'implementazione.** Al fine di semplificare la definizione e l'esecuzione dei test case sono state sviluppate **6 funzioni di utilità**. Tali funzioni sono state a loro volta testate, ma i relativi dettagli sono omessi dal presente documento e non sono conteggiati nei totali di tabella 38.

### 7.2.3 UC-CR3 — Consultazione delle analisi tramite dashboard

**Tabella 33:** UC-CR3 — Consultazione della dashboard

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrizione</b>                 | L'utente accede alla tab Dashboard della GUI, seleziona una delle analisi disponibili e ne visualizza i risultati in forma grafica e aggregata (overview, metriche e top librerie).                                                                                                                                                                                                                        |
| <b>Attori</b>                      | Utente (ricercatore)                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Entry condition</b>             | GUI aperta; utente nella tab Dashboard; interfaccia in stato Idle.                                                                                                                                                                                                                                                                                                                                         |
| <b>Exit condition</b>              | <i>Successo</i> : analisi selezionata visualizzata correttamente; interfaccia in stato Idle sulla tab Dashboard.<br><i>Errore</i> : errore imprevisto o visualizzazione fallita; interfaccia in stato Idle sulla tab Dashboard con messaggio di errore mostrato.                                                                                                                                           |
| <b>Flusso di eventi principale</b> | <ol style="list-style-type: none"> <li>1. Il sistema mostra l'elenco delle analisi disponibili e attende una selezione.</li> <li>2. L'utente seleziona un'analisi (ad es. <code>Analysis_1</code>).</li> <li>3. Il sistema visualizza Analysis Overview (conteggi e percentuali Producer/Consumer/Producer &amp; Consumer), Code Quality Metrics (CC e MI medi) e Top 10 ML Libraries Detected.</li> </ol> |

**Parametri:** `io/output/producer` e `io/output/consumer` (percorsi contenenti i risultati delle analisi per producer e consumer); `io/output/metrics` (percorso contenente i risultati per le metriche CC e MI). **Oggetti dell’ambiente:** stato del filesystem.

**Tabella 34:** UC-CR3 — Categorie e scelte

| Categoria                     | ID   | Descrizione                                                    | Proprietà               |
|-------------------------------|------|----------------------------------------------------------------|-------------------------|
| Directory producer e consumer | DPC0 | Directory vuote (0 risultati per progetti producer e consumer) | [property DirProConEmp] |
|                               | DPC1 | Directory con 1+ risultati per progetti producer e consumer    | [property DirProMulPro] |
| Directory metrics             | DM0  | Directory vuota (0 progetti analizzati)                        | [property DirMetEmp]    |
|                               | DM1  | Directory con 1+ risultati per progetti analizzati             | [property DirMetMulPro] |

**Tabella 35:** UC-CR3 — Test frame

| ID  | Combinazioni | Oracolo (risultato atteso)                                                                                                                             |
|-----|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| TF1 | DPC0, DM0    | Non compare nessuna Analysis                                                                                                                           |
| TF2 | DPC0, DM1    | Compare Analysis; selezionandola mostra la media delle metriche dei progetti in input, mentre i restanti valori sono pari a 0                          |
| TF3 | DPC1, DM0    | Compare Analysis; selezionandola mostra il numero di consumer e producer corrispondenti e le keyword più usate, mentre i restanti valori sono pari a 0 |
| TF4 | DPC1, DM1    | Compare Analysis; selezionandola mostra numero di consumer e producer, le keyword più usate e la media delle metriche                                  |

I quattro test frame sono stati istanziati in altrettanti test case, implementati in `test/system_testing/dashboard_test/dashboard_test.py`. Ogni test case popola le directory di output con CSV controllati, apre la dashboard e verifica le etichette del Classification Summary, i valori di Code Metrics e il contenuto della tabella ML Keywords Usage.

### 7.3 Regression Testing

Il regression testing è stato condotto tramite la riesecuzione completa della suite di test preesistente, in particolare i test di sistema eseguiti nella fase pre-modifica (sezione 3): **16 test case** definiti sui due casi d’uso principali (UC-1 Analysis: 12 test case; UC-2 Cloning: 4 test case) e completati senza fallimenti prima delle modifiche.

**Tabella 36:** Riepilogo della nuova esecuzione della suite di regressione

| Totale | Superati | Falliti |
|--------|----------|---------|
| 16     | 16       | 0       |

L’esito conferma sul campo il vincolo progettuale enunciato in sezione 6.3: le funzionalità preesistenti si comportano esattamente come prima delle modifiche.

## 7.4 Riepilogo dei risultati

Al termine dell'esecuzione dei vari test, tutti gli **84 casi di test** previsti sono stati eseguiti con successo, senza riscontrare alcun fallimento.

**Tabella 37:** Risultati complessivi

| Test eseguiti | Superati | Fallimenti | Incident /<br>anomalie |
|---------------|----------|------------|------------------------|
| 84            | 84       | 0          | Nessuna                |

**Tabella 38:** Ripartizione dei test eseguiti

| Fase                               | Tipo                           | Riferimento                     | Numero        |    |
|------------------------------------|--------------------------------|---------------------------------|---------------|----|
| Test eseguiti<br>post-modification | Unit Test                      | CR1                             | 8             |    |
|                                    |                                | CR2                             | 8             |    |
|                                    |                                | CR3                             | 2             |    |
|                                    | Integration Test               | CR1                             | 8             |    |
|                                    |                                | CR2                             | 13            |    |
|                                    |                                | CR3                             | 7             |    |
|                                    | System Test (CR1, CR2,<br>CR3) | CR1                             | 6             |    |
|                                    |                                | CR2                             | 12            |    |
|                                    |                                | CR3                             | 4             |    |
|                                    | Regression Testing             | System Test<br>Pre-Modification | UC-1 Analysis | 12 |
|                                    |                                |                                 | UC-2 Cloning  | 4  |
| Totale                             |                                |                                 | 84            |    |

**Corrispondenza con la suite automatizzata.** Eseguendo `python -m pytest` l'intera suite raccoglie **90** test: gli 84 casi di test documentati qui più le **6** funzioni di utilità del `gui_test` citate in sezione 7.2.2, che non costituiscono test case del piano ma sono comunque verificate.

## 8 Conclusioni

### 8.1 Sintesi dei risultati

Il lavoro ha portato MARK 2.0 a MARK 2.0 Plus attraverso tre Change Request, tutte rilasciate e verificate. I risultati principali sono riassunti in tabella 39, che rimanda ai dati di dettaglio riportati nelle sezioni precedenti anziché ripeterli.

**Tabella 39:** Risultati chiave dell'evoluzione MARK 2.0 Plus

| Risultato                                                | Descrizione                                                                                                                                                            | Riferimento            |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| Integrazione strutturata delle metriche di qualità (CR1) | Il core è stato esteso con il calcolo automatico di Cyclomatic Complexity e Maintainability Index tramite Radon [8], con aggregazione a livello di progetto.           | sezione 4.2            |
| Introduzione di un layer di configurazione (CR2)         | La GUI sviluppata in Tkinter consente di configurare ed eseguire l'analisi senza ricorrere alla CLI e senza modificare manualmente i parametri all'interno del codice. | sezione 4.3            |
| Dashboard grafica integrata (CR3)                        | I risultati sono presentati tramite grafici dinamici (matplotlib) che rendono immediata l'interpretazione dei risultati.                                               | sezione 4.4            |
| Impatto stimato e poi misurato                           | L'Impact Analysis della CR1 ha ottenuto recall pari a 1 con precision 0,77; per CR2 e CR3 l'impatto sul core è nullo per costruzione.                                  | tabella 20             |
| Elevata copertura di test                                | Le componenti modificate raggiungono il 97 % di branch coverage nello Unit Testing e il 90 % nell'Integration Testing.                                                 | tabella 24             |
| Baseline verificata e stabile                            | Dai due casi d'uso principali è stata derivata una suite di 16 system test, rieseguita dopo le modifiche senza alcun fallimento.                                       | tabella 11, tabella 36 |

Complessivamente sono stati eseguiti **84 casi di test** documentati, tutti superati e senza incident (tabella 37).

## 8.2 Valutazione del processo

Al di là delle funzionalità aggiunte, il progetto ha consolidato un processo di **evoluzione controllata** basato su change management, impact analysis e testing sistematico. Tre osservazioni meritano di essere messe in evidenza.

**I falsi positivi come misura dell'estensibilità.** Il risultato metodologicamente più interessante riguarda l'Impact Analysis della CR1. I tre falsi positivi (`MLAnalysisFacade.run_analysis`, `AnalyzerFactory.create_builder`, `AnalyzerBuilder.build`) non sono il sintomo di una stima imprecisa: sono la conferma empirica che i design pattern già presenti nella baseline — in particolare il Factory-Registry — erano sufficientemente estensibili da assorbire un nuovo ruolo di analisi senza alcuna modifica. Una previsione ragionevole ha indicato quei metodi come candidati; il codice ha dimostrato che non serviva toccarli.

**Il valore di un'analisi conservativa.** La scelta di espandere il call graph sia in fan-in sia in fan-out ha prodotto una sovra-approssimazione voluta: precision 0,77 a fronte di recall pari a 1 (equazioni (3) e (4)). In manutenzione è il compromesso desiderabile, perché i due errori non hanno lo stesso costo — un falso positivo costa una verifica in più, un falso negativo costa un difetto non individuato.

**Un vincolo progettuale verificato, non solo dichiarato.** CR2 e CR3 sono state progettate come strati che invocano il core senza modificarlo, il che rende l'impatto sui componenti preesistenti nullo *per costruzione* (sezione 6.3). Il punto è che tale vincolo non è rimasto un'affermazione di progetto: la suite di 16 test di sistema, definita *prima* di qualunque modifica proprio a questo scopo, lo ha confermato sperimentalmente al termine del lavoro.



### 8.3 Limiti e sviluppi futuri

Il principale limite riguarda la copertura di `gui/controller.py`, che si ferma al 68 % di branch coverage in integrazione e resta quindi **sotto l'obiettivo di  $\geq 80\%$**  fissato dal Master Test Plan (sezione 5.2.3); le altre tre classi target superano ampiamente la soglia. Il completamento della copertura sui rami di gestione degli errori del Controller è la prima attività di verifica da pianificare.

Sul piano funzionale si prevede di:

- estendere il numero di tipologie di classificazione ML (oltre Producer/Consumer);
- integrare nuove metriche di qualità (ad es. coupling, cohesion);
- arricchire la GUI con nuovi grafici e filtri interattivi.

## 9 Riferimenti bibliografici

- [1] Priyadarshi Tripathy e Kshirasagar Naik. *Software Evolution and Maintenance: A Practitioner's Approach*. Riferimento per gli SLO (Software Life-cycle Objects) e per l'Impact Analysis basata su SIS, CIS, AIS, FPIS e DIS. John Wiley & Sons, 2014. ISBN: 978-0-470-60341-3.
- [2] Thomas J. McCabe. "A Complexity Measure". In: *IEEE Transactions on Software Engineering* SE-2.4 (1976), pp. 308–320. DOI: 10.1109/TSE.1976.233837.
- [3] Don Coleman, Dan Ash, Bruce Lowther e Paul Oman. "Using Metrics to Evaluate Software System Maintainability". In: *Computer* 27.8 (1994), pp. 44–49. DOI: 10.1109/2.303623.
- [4] Thomas J. Ostrand e Marc J. Balcer. "The Category-Partition Method for Specifying and Generating Functional Tests". In: *Communications of the ACM* 31.6 (1988), pp. 676–686. DOI: 10.1145/62959.62964.
- [5] Jesus M. Gonzalez-Barahona e Gregorio Robles. "On the Reproducibility of Empirical Software Engineering Studies Based on Data Retrieved from Development Repositories". In: *Empirical Software Engineering*. Vol. 17. Contesto sulla costruzione di dataset affidabili a partire da repository di sviluppo. 2012, pp. 75–89. DOI: 10.1007/s10664-011-9181-9.
- [6] *ISO/IEC/IEEE 14764:2022 — Software engineering — Software life cycle processes — Maintenance*. Definisce la Modification Request e la sua classificazione in Correction ed Enhancement, con le tipologie di manutenzione corrective, preventive, adaptive, additive e perfective. Geneva, CH: International Organization for Standardization, 2022.
- [7] *pytest-cov — Coverage plugin for pytest*. <https://pytest-cov.readthedocs.io>. Utilizzato per la misura della branch coverage. 2024.
- [8] Michele Lacchia. *Radon — Various code metrics for Python code*. <https://radon.readthedocs.io>. Libreria utilizzata per il calcolo di Cyclomatic Complexity e Maintainability Index. 2024.